



LLMs - RAG & Deep IR

Benjamin Piwowarski

Lecture 5

<https://master-mind.isir.upmc.fr/llm/slides>

Press ? for help on keyboard shortcuts

[Slides in PDF](#)



Machine Learning &
Deep Learning for
Information Access

TODAY'S LECTURE

1. Retrieval-Augmented Generation (RAG)

- Motivation: addressing hallucinations
- Architecture and components

2. Information Retrieval Foundations

- Traditional IR: BM25, TF-IDF
- Evaluation metrics: Precision, Recall, nDCG

3. Neural Information Retrieval

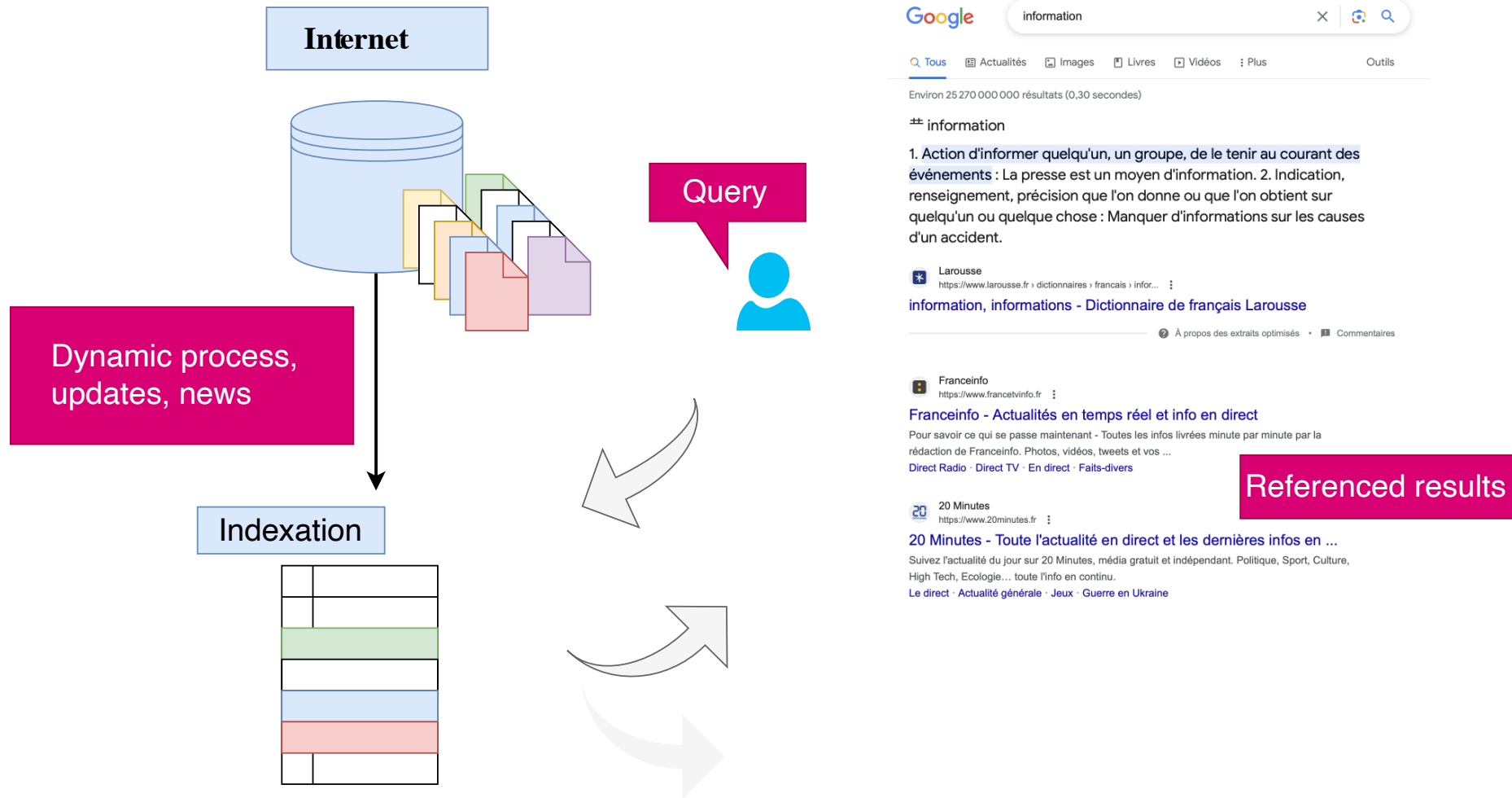
- Dense models (bi-encoders) and vector indexing
- Interaction models (cross-encoders, ColBERT)
- Learned sparse models (SPLADE) and inverted indices

4. Learning to Rank

- Loss functions and training strategies

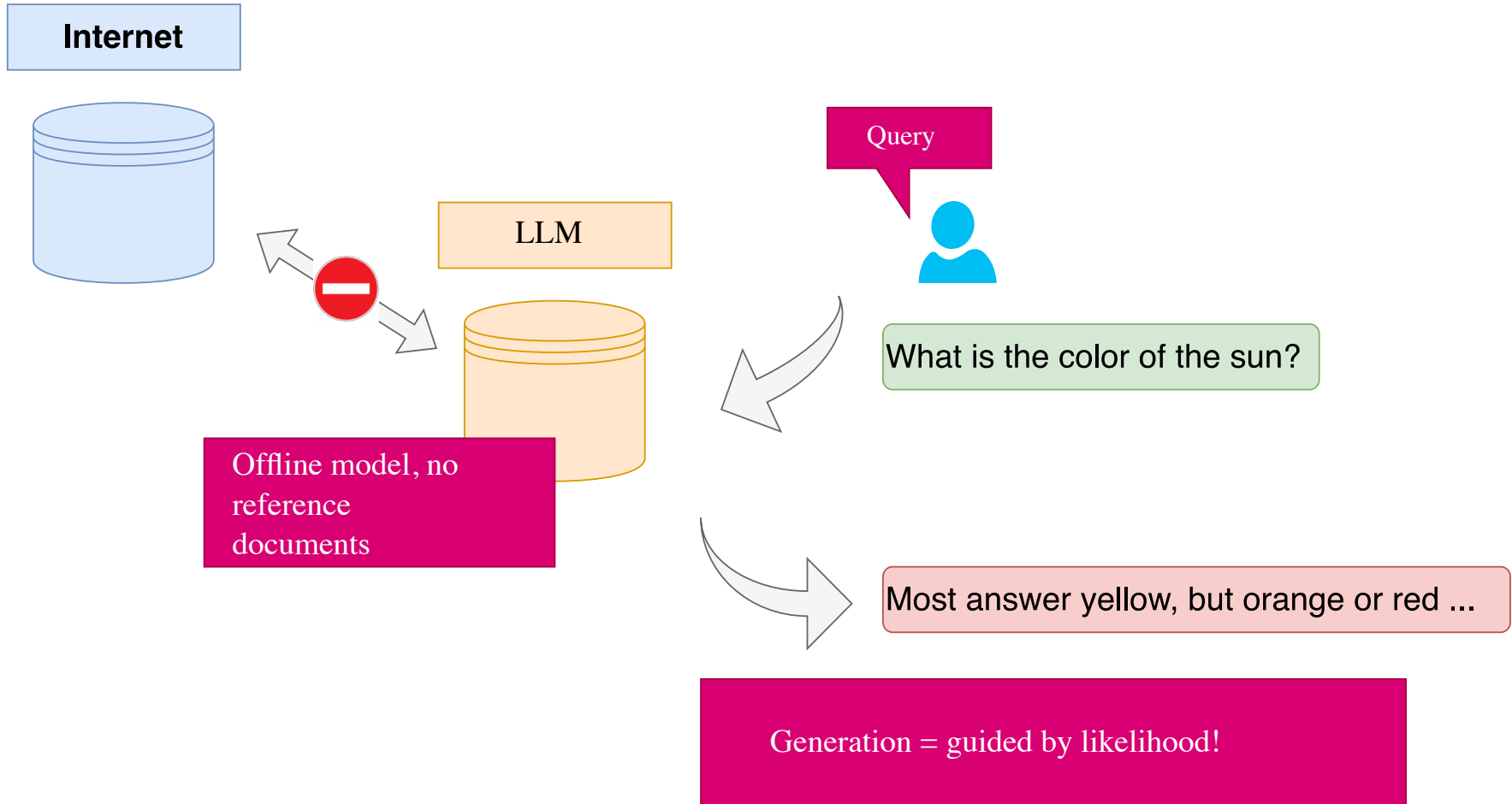
RAG: RETRIEVAL-AUGMENTED GENERATION

RAG: RETRIEVAL-AUGMENTED GENERATION ► WITH A SEARCH ENGINE



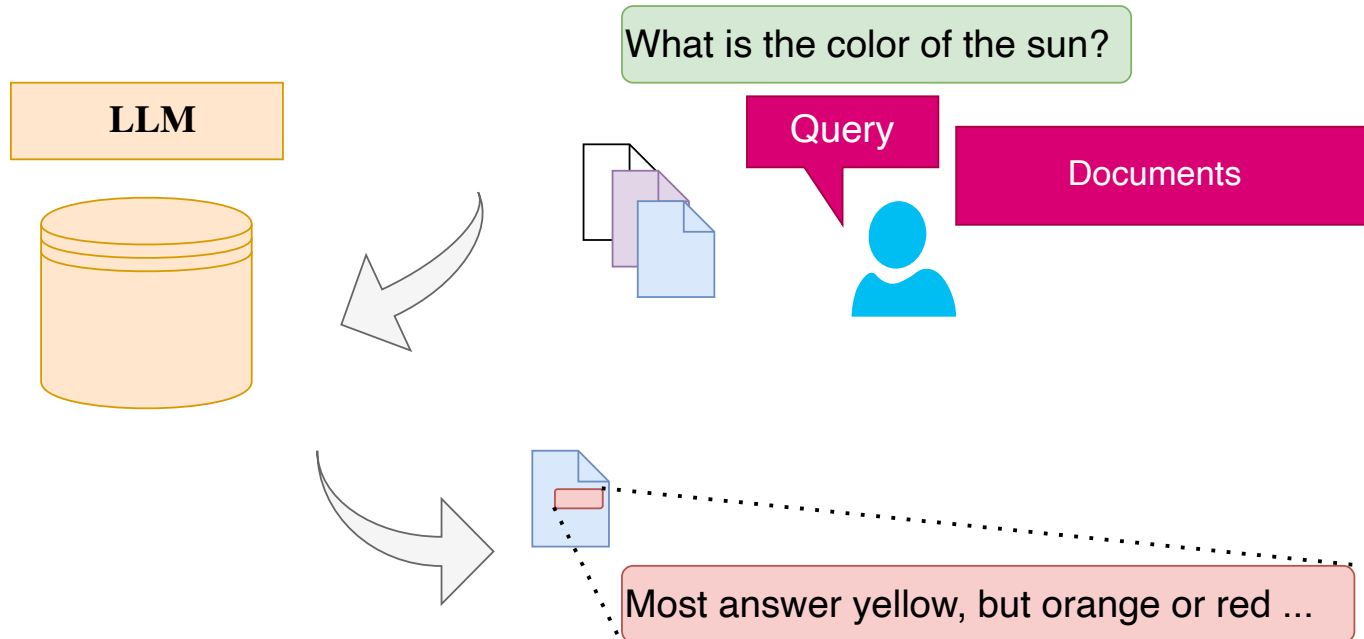
RAG: RETRIEVAL-AUGMENTED GENERATION ► LLM: INFORMATION ACCESS VIA DIALOGUE

- 🔑 Specialized responses based on retrieved context
- 🔑 Many applications beyond simple Q&A...



RAG: RETRIEVAL-AUGMENTED GENERATION ► DOCUMENT ANALYSIS

- 🔑 Question Answering architecture
- 🔑 Document analysis / summarization
- 🔑 Meeting transcripts and reports



⚠ LLMs Make Things Up

- Generate plausible but incorrect information
- Confidently state outdated facts
- Cannot access real-time information
- Knowledge cutoff limits usefulness

⚠ LLMs Make Things Up

- Generate plausible but incorrect information
- Confidently state outdated facts
- Cannot access real-time information
- Knowledge cutoff limits usefulness

Solution ?

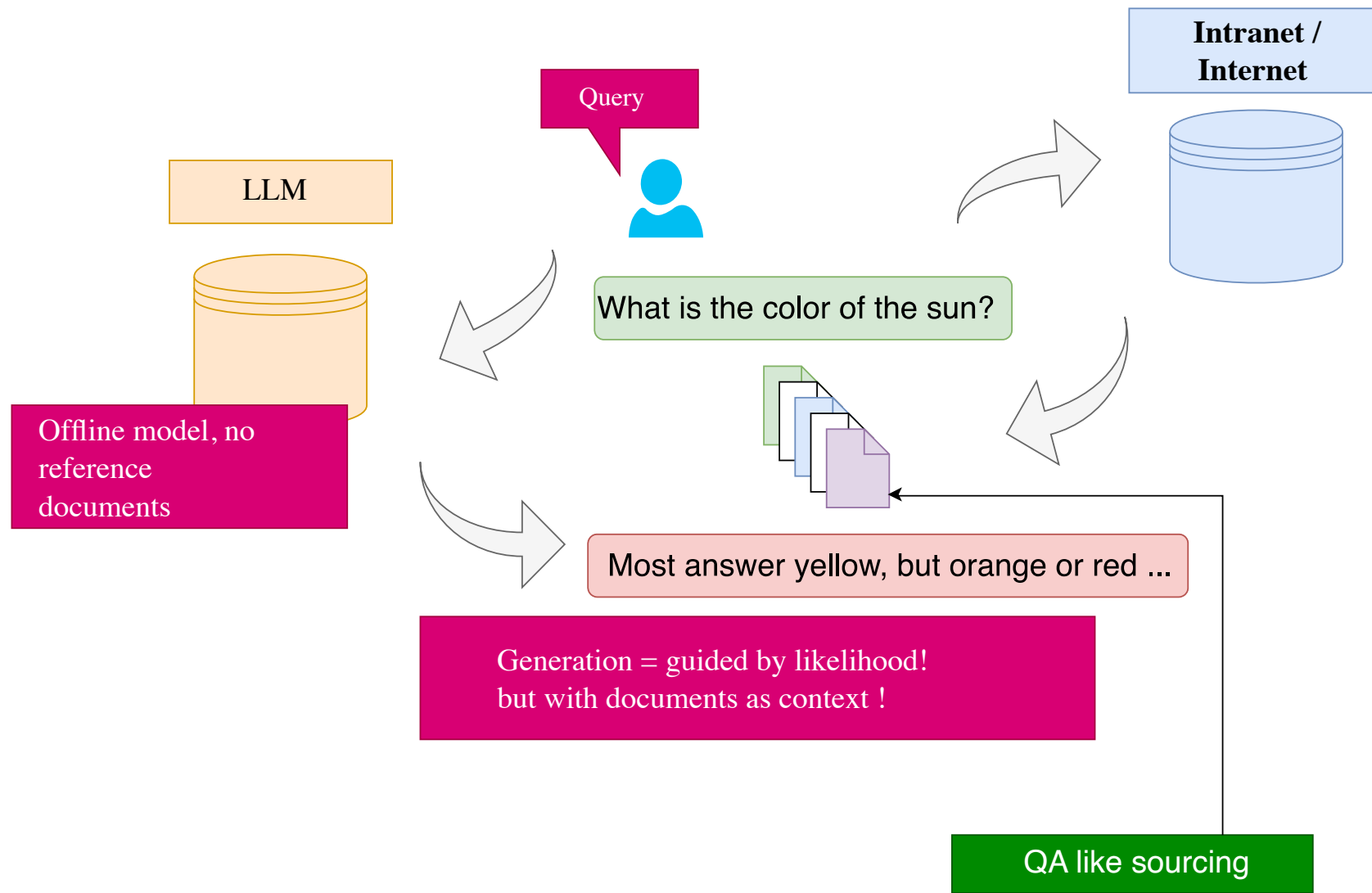
Augment generation with retrieved information!

RAG: RETRIEVAL-AUGMENTED GENERATION RAG FRAMEWORK

RAG: RETRIEVAL-AUGMENTED GENERATION ► RETRIEVAL-AUGMENTED GENERATION (RAG)

Web search + document retrieval + LLM generation

RAG: Retrieval Augmented Generation



► RAG Architecture

1. **Query** - User question or prompt
2. **Retrieve** - Find relevant documents from corpus
3. **Augment** - Add retrieved context to prompt
4. **Generate** - LLM produces answer grounded in context

📌 RAG Architecture

1. **Query** - User question or prompt
2. **Retrieve** - Find relevant documents from corpus
3. **Augment** - Add retrieved context to prompt
4. **Generate** - LLM produces answer grounded in context

💡 Key Benefit

Grounds generation in retrieved facts, reducing hallucinations



Augmented Prompt

Use the following context to answer the question.

Context:

- [1] The Eiffel Tower is 330 meters tall and was completed in 1889 for the World's Fair.
- [2] It receives about 7 million visitors annually.

Question: How tall is the Eiffel Tower?

Answer based only on the provided context.

Augmented Prompt

Use the following context to answer the question.

Context:

- [1] The Eiffel Tower is 330 meters tall and was completed in 1889 for the World's Fair.
- [2] It receives about 7 million visitors annually.

Question: How tall is the Eiffel Tower?

Answer based only on the provided context.

Without RAG

LLM might hallucinate: *"The Eiffel Tower is 324 meters tall"*

With RAG

LLM answers: *"The Eiffel Tower is 330 meters tall [1]"*

CORE COMPONENTS

Retriever

Finds relevant documents (sparse, dense, or hybrid)

Document Store

Index of documents (vector DB, search engine)

Generator

LLM that produces the final response

Prompt Template

Combines query and retrieved context

RAG: RETRIEVAL-AUGMENTED GENERATION ► DOES BETTER RETRIEVAL MEAN BETTER ANSWERS?

The Assumption ?

Improving retrieval quality should improve answer quality... right?

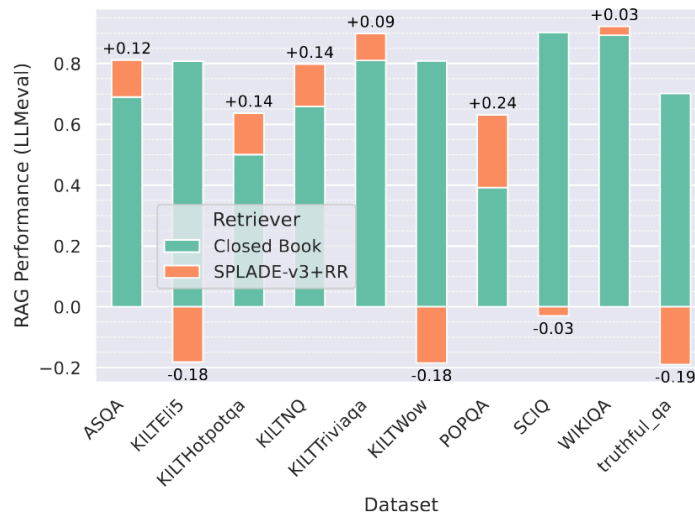
RAG: RETRIEVAL-AUGMENTED GENERATION ► DOES BETTER RETRIEVAL MEAN BETTER ANSWERS?

The Assumption ?

Improving retrieval quality should improve answer quality... right?

RAG has a strong impact on answer quality

 Rau, D. et al. 2024. BERGEN: A Benchmarking Library for Retrieval-Augmented Generation. Findings of the Association for Computational Linguistics: EMNLP 2024.



Most tasks are impacted positively, but... not always!

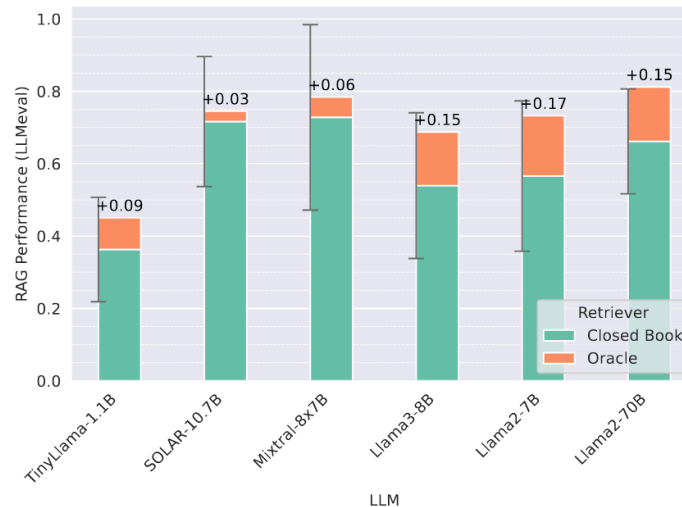
RAG: RETRIEVAL-AUGMENTED GENERATION ► DOES BETTER RETRIEVAL MEAN BETTER ANSWERS?

The Assumption ?

Improving retrieval quality should improve answer quality... right?

All LLMs can be positively impacted

 Rau, D. et al. 2024. BERGEN: A Benchmarking Library for Retrieval-Augmented Generation. Findings of the Association for Computational Linguistics: EMNLP 2024.



All LLMs benefit from RAG, even strong ones (LLama 2-70B).

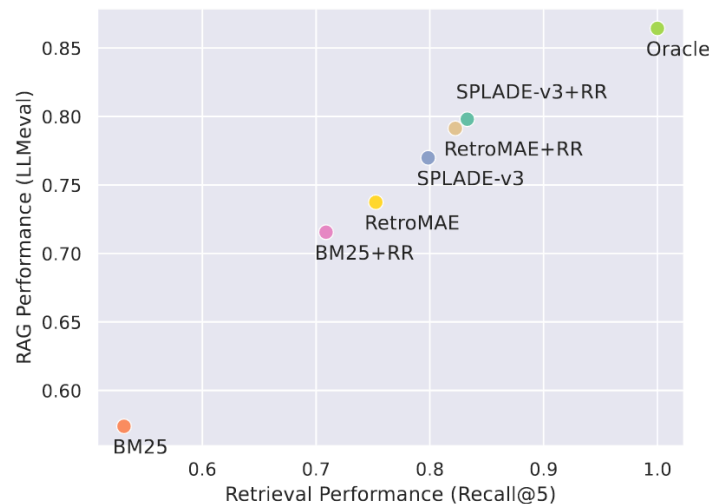
RAG: RETRIEVAL-AUGMENTED GENERATION ► DOES BETTER RETRIEVAL MEAN BETTER ANSWERS?

The Assumption ?

Improving retrieval quality should improve answer quality... right?

Strong correlation between retrieval and answer quality

 Rau, D. et al. 2024. BERGEN: A Benchmarking Library for Retrieval-Augmented Generation. Findings of the Association for Computational Linguistics: EMNLP 2024.



There is a clear correlation between retrieval quality (Recall@5) and RAG performance

USE RAG WHEN

- Need up-to-date information
- Domain-specific knowledge required
- Factual accuracy is critical
- Auditability needed (cite sources)

CONSIDER ALTERNATIVES WHEN

- General knowledge is sufficient
- Latency is critical
- No relevant corpus available
- Creative generation needed

TRADITIONAL INFORMATION RETRIEVAL

TRADITIONAL INFORMATION RETRIEVAL

IR FUNDAMENTALS

Definition

"Information retrieval (IR) is finding material (usually documents) of an unstructured nature (usually text) that satisfies an information need from within large collections (usually stored on computers)."

— C. Manning

- ☞ Also in enterprises, digital libraries, personal computers...
- ☞ Specialized domains: medicine, law, scientific literature
- ☞ Most common application: **search engines**

Google YAHOO!

Yandex AOL. Ask.com

Baidu 百度 Bing

Information is available

BUT: the problem lies in *selection*

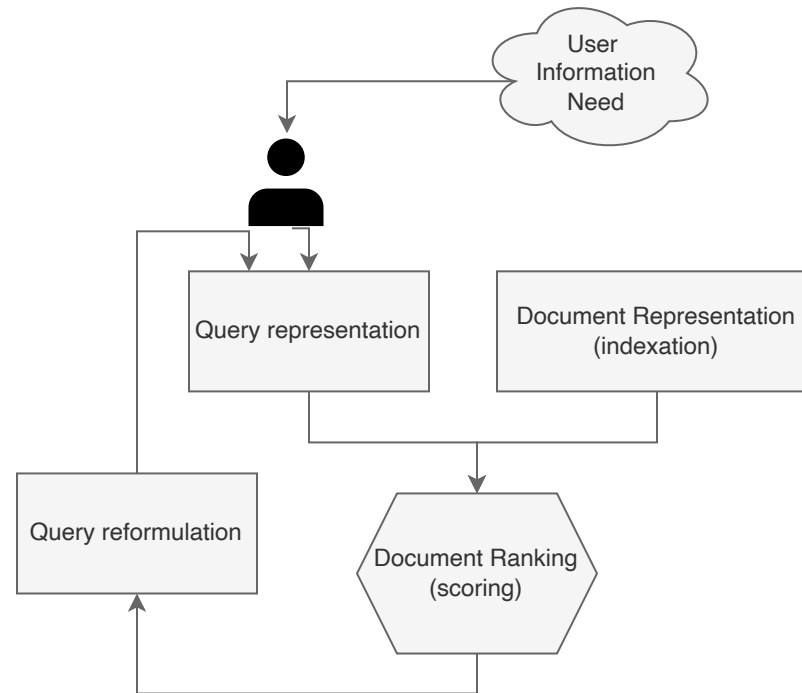
Finding the right information for the right user at the right time



Indexing: Extract document content into an index; same process for queries to enable matching

Matching: Relate the indexed document collection with the processed query to identify relevant documents

Query Reformulation: Refine the user's information need throughout the search session



Hypotheses Behind Document Matching

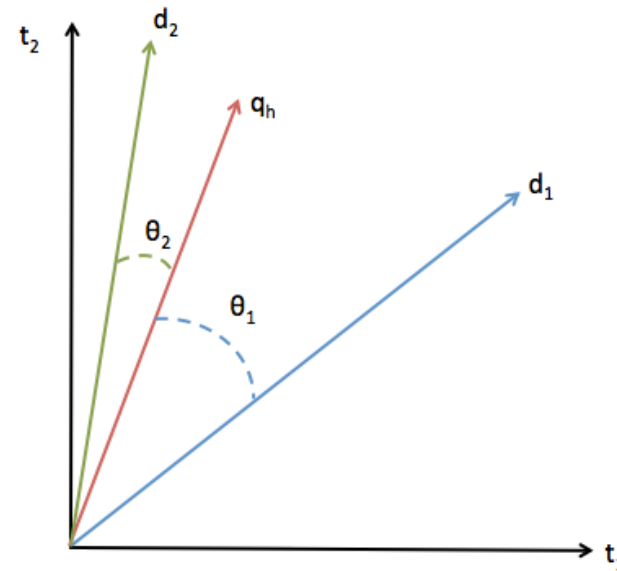
The more words query and document share, the more relevant the document

The more similar the term distributions, the more relevant the document

TRADITIONAL INFORMATION RETRIEVAL

CLASSICAL MODELS

- Feature space: terms $t_i, i = 1 \dots n$ (preprocessed)
- Documents and queries represented as weight vectors:
 - Document: $d = (x_0, \dots, x_{n-1})$
 - Query: $q = (y_0, \dots, y_{n-1})$
- x_k = weight of term k in document d :
 - Presence/absence (binary)
 - Term frequency (TF)
 - TF-IDF (most common)



SIMILARITY MEASURES

INNER PRODUCT

$$s(q, d) = q \cdot d = \sum_i x_i \cdot y_i$$

COSINE SIMILARITY

$$s(q, d) = \frac{q \cdot d}{||q|| \cdot ||d||}$$

KEY IDEAS

- Extends the Binary Independence Model (BIM)
- **Eliteness**: a term is "elite" if it represents the document
 - Elite terms follow Poisson distribution with parameter λ_t
 - Non-elite terms follow Poisson with $\mu_t < \lambda_t$
- The more frequent a term, the more likely it is elite

BM25 FORMULA

$$s(q, d) = \sum_{t \in q \cap d} \underbrace{\log \frac{N - n_t + 0.5}{n_t + 0.5}}_{IDF} \times \underbrace{\frac{tf_t}{tf_t + k_1 \cdot B}}_{TF \text{ saturation}}$$

where $B = (1 - b) + b \cdot \frac{|d|}{avgdl}$ (document length normalization)

PARAMETERS

- $k_1 \approx 1.2$: saturation parameter
- $b \approx 0.75$: length normalization

INTUITION FOR b

- $b = 1$: verbose text (term repeats because doc is long)
- $b = 0$: multi-topic text (term repeats because important)

TRADITIONAL INFORMATION RETRIEVAL EVALUATION METRICS

Relevance

A document is relevant if it satisfies the user's information need.

CONTINGENCY TABLE

	Retrieved	Not Retrieved
Relevant	TP (true positive)	FN (false negative)
Not Relevant	FP (false positive)	TN (true negative)

- R = set of relevant documents
- A = set of retrieved documents (answer)

BINARY RELEVANCE METRICS

PRECISION

Fraction of retrieved that are relevant:

$$P = \frac{|R \cap A|}{|A|} = \frac{TP}{TP + FP}$$

F1 SCORE

Harmonic mean:

$$F_1 = \frac{2PR}{P+R} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

RECALL

Fraction of relevant that are retrieved:

$$R = \frac{|R \cap A|}{|R|} = \frac{TP}{TP + FN}$$

AVERAGE PRECISION (AP)

Average of precision at each relevant document position

$$AP = \frac{1}{|R|} \sum_{k=1}^n P@k \cdot \text{rel}(k)$$

where $\text{rel}(k) = 1$ if document at rank k is relevant, 0 otherwise

MEAN AVERAGE PRECISION (MAP)

Mean of AP across queries

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} AP(q)$$

NORMALIZED DISCOUNTED CUMULATIVE GAIN (NDCG)

Accounts for graded relevance and position discount

$$nDCG@k = \frac{DCG@k}{IDCG@k} = \frac{\sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i+1)}}{IDCG@k}$$

NEURAL INFORMATION RETRIEVAL

Information Retrieval (IR)

Finding documents relevant to an information need expressed as a query

THE IR TASK

INPUT

- A collection of documents
 $\mathcal{D} = \{d_1, \dots, d_n\}$
- A query q (user's information need)

OUTPUT

- A ranked list of documents
- Ordered by decreasing relevance to q

KEY CHALLENGES

- **Scale:** Collections with millions/billions of documents
- **Speed:** Sub-second response time required
- **Relevance:** Understanding semantic meaning, not just keywords

CLASSICAL IR (1960S-2010S)

- Bag-of-words: TF-IDF, BM25
- Exact term matching
-  Fast and scalable (inverted index)
-  Vocabulary mismatch problem ("car" vs "automobile")

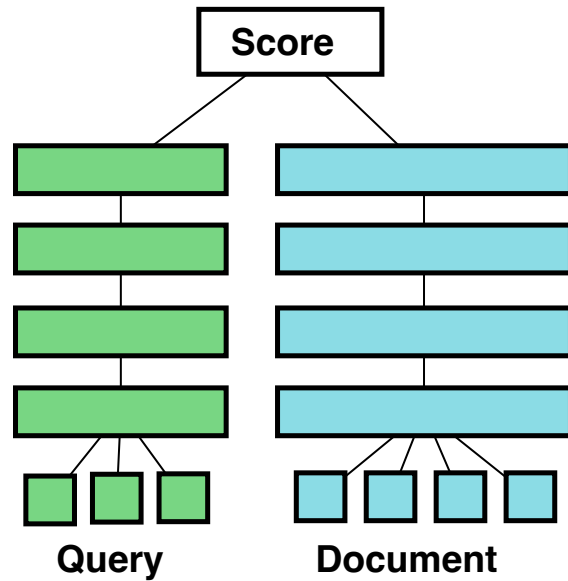
NEURAL IR (2013-NOW)

- Semantic matching beyond exact keywords
-  Better relevance (understands meaning)
-  Computationally expensive

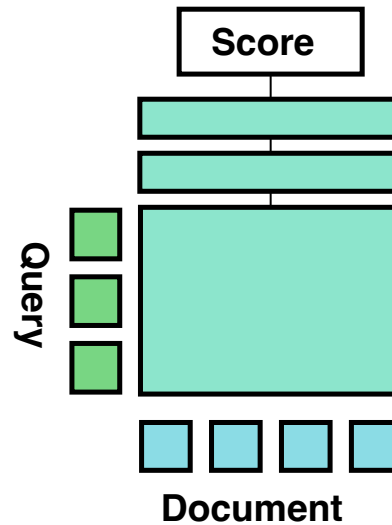
KEY QUESTION

- 🔑 How to combine the **efficiency** of classical IR with the **effectiveness** of neural models?
- 🔑 How to learn neural models that generalize well to other collections than the one used for training?

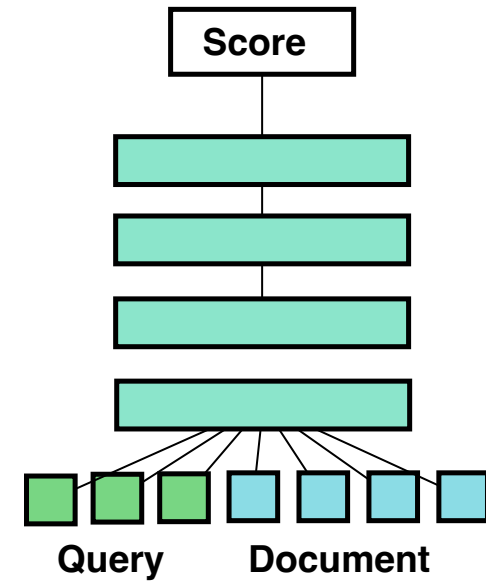
REPRESENTATION BASED



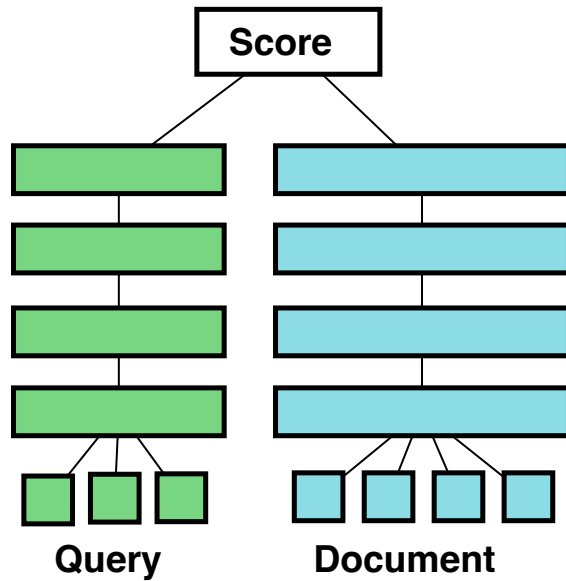
LATE-INTERACTION



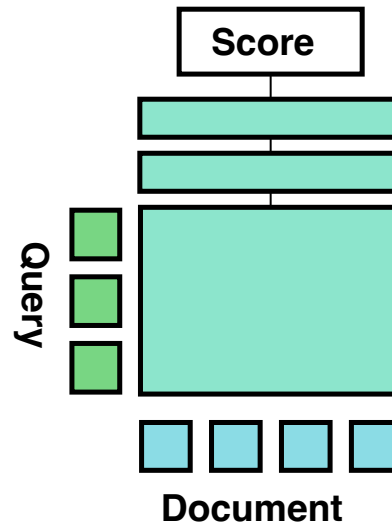
EARLY-INTERACTION



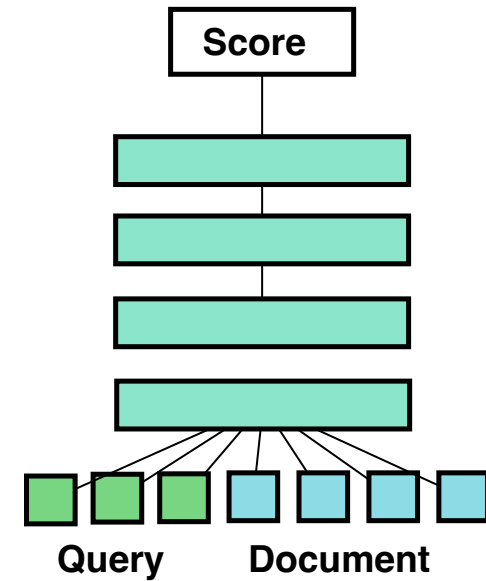
REPRESENTATION BASED



LATE-INTERACTION

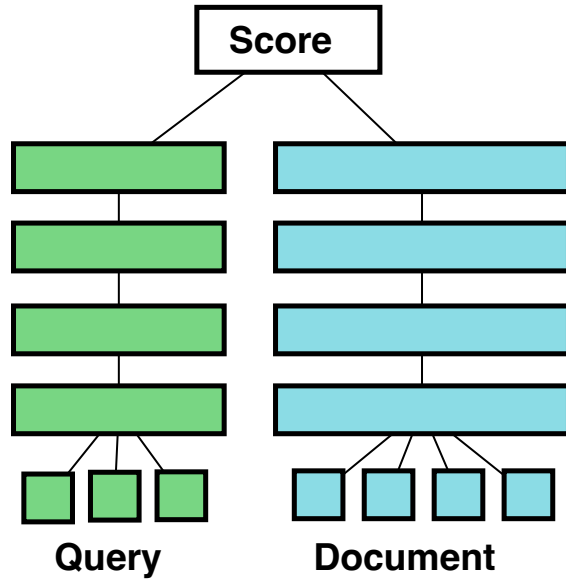


EARLY-INTERACTION



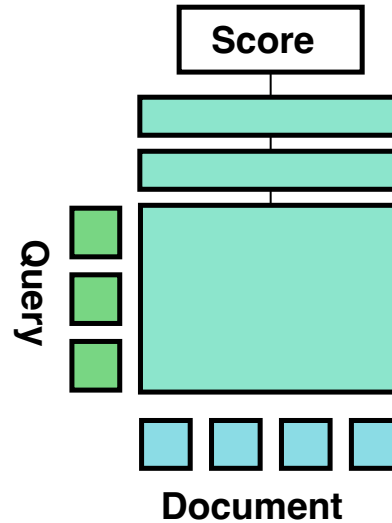
A score is computed from fixed document and query embeddings

REPRESENTATION BASED



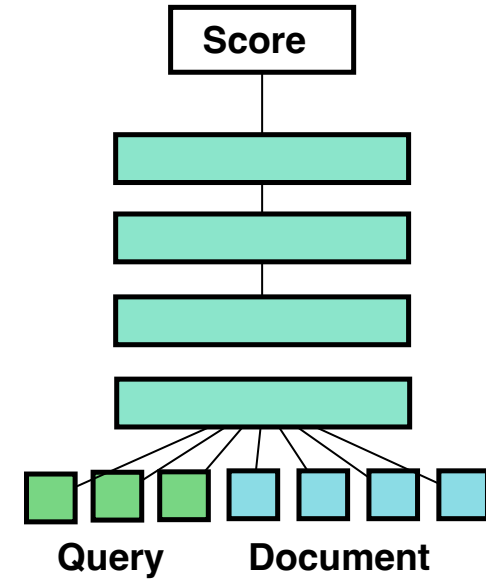
A score is computed from fixed document and query embeddings

LATE-INTERACTION

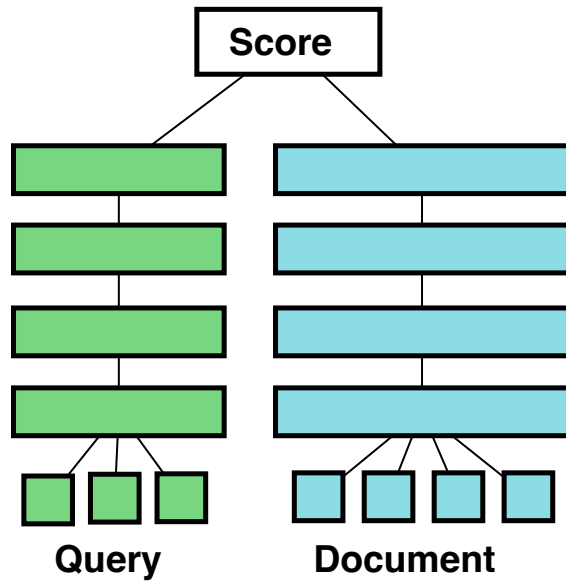


A score is computed for each pair of query and document **token** embeddings. Those pairwise scores are aggregated into a final query-document score.

EARLY-INTERACTION

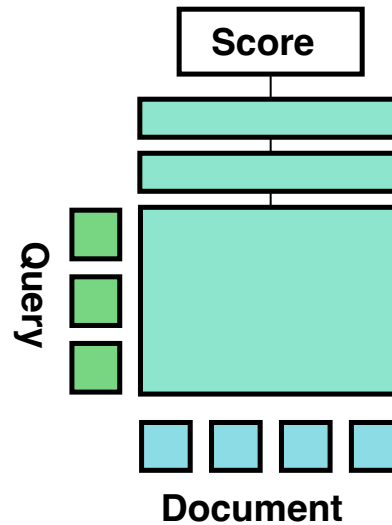


REPRESENTATION BASED



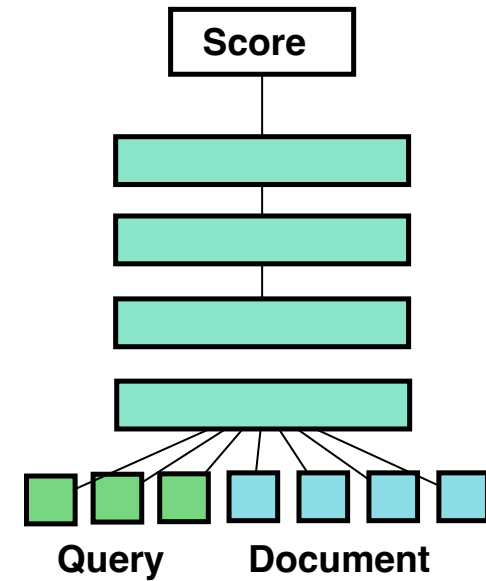
A score is computed from fixed document and query embeddings

LATE-INTERACTION



A score is computed for each pair of query and document **token** embeddings. Those pairwise scores are aggregated into a final query-document score.

EARLY-INTERACTION



Uses the full interaction between query and document tokens to compute a relevance score.

PIPELINE ARCHITECTURE

- 1 First Stage:** Fast retrieval (BM25, dense bi-encoder)
Retrieve top-K candidates ($K = 100-1000$)
🔑 Rely on efficient indices (inverted index, ANN index)
- 2 Second Stage:** Re-ranking (cross-encoder)
Score all K candidates with expensive model
- 3 Optional Third Stage:** Diversification, filtering
Remove duplicates, apply business rules

PIPELINE ARCHITECTURE

- 1 **First Stage:** Fast retrieval (BM25, dense bi-encoder)
 Retrieve top-K candidates ($K = 100-1000$)
 ☞ Rely on efficient indices (inverted index, ANN index)
- 2 **Second Stage:** Re-ranking (cross-encoder)
 Score all K candidates with expensive model
- 3 **Optional Third Stage:** Diversification, filtering
 Remove duplicates, apply business rules

💡 Why Multi-Stage?

Cross-encoders are accurate but expensive
 First stage reduces candidates to manageable size

NEURAL INFORMATION RETRIEVAL

DENSE TEXT REPRESENTATIONS

💡 Semantic Space

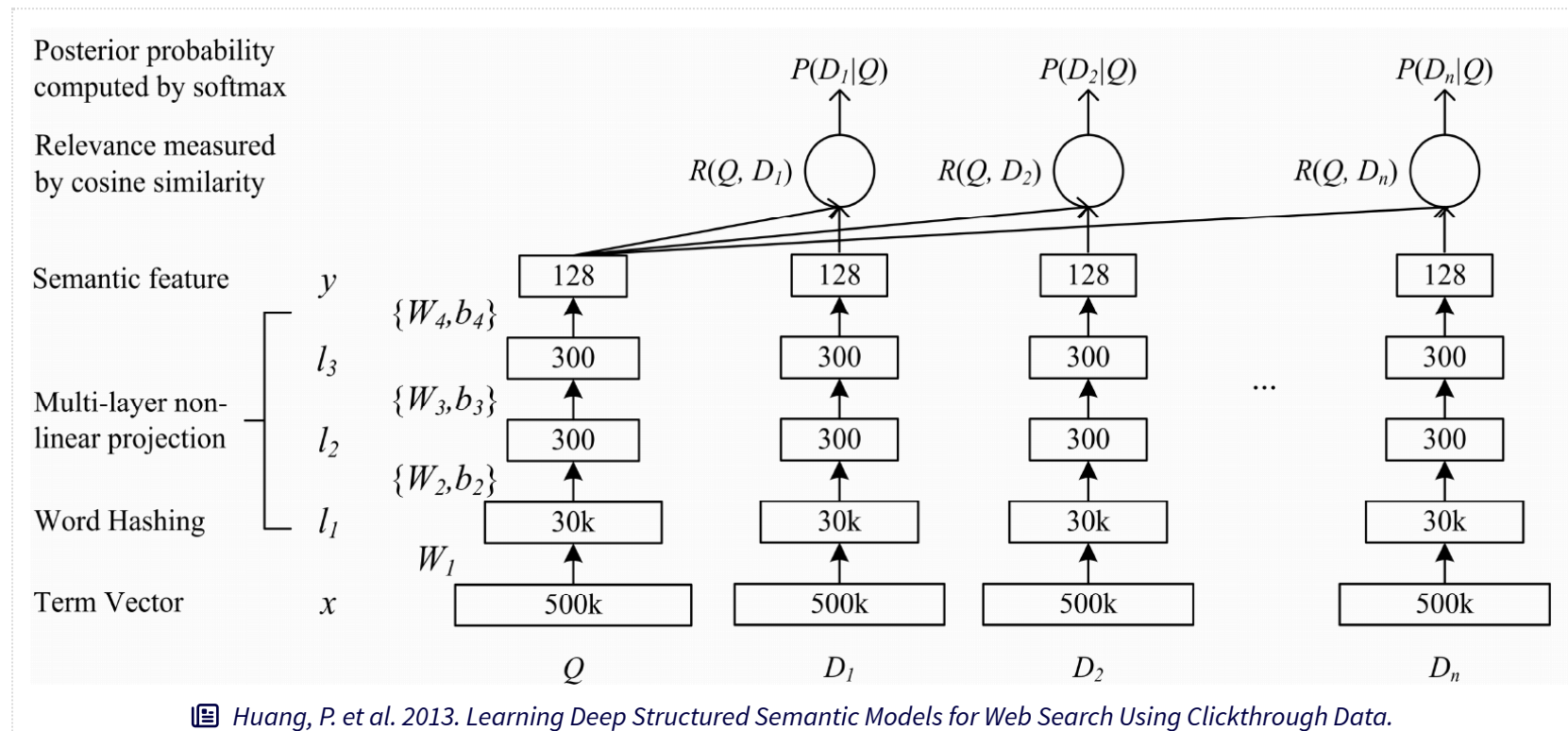
Every text t is represented by $f_{\theta}(t) \in \mathbb{R}^n$

- Symmetric model: $s(t_1, t_2) = g(f_{\theta}(t_1), f_{\theta}(t_2))$
- Asymmetric model: $s(t_1, t_2) = g(f_{\theta}^{(q)}(t_1), f_{\theta}^{(d)}(t_2))$

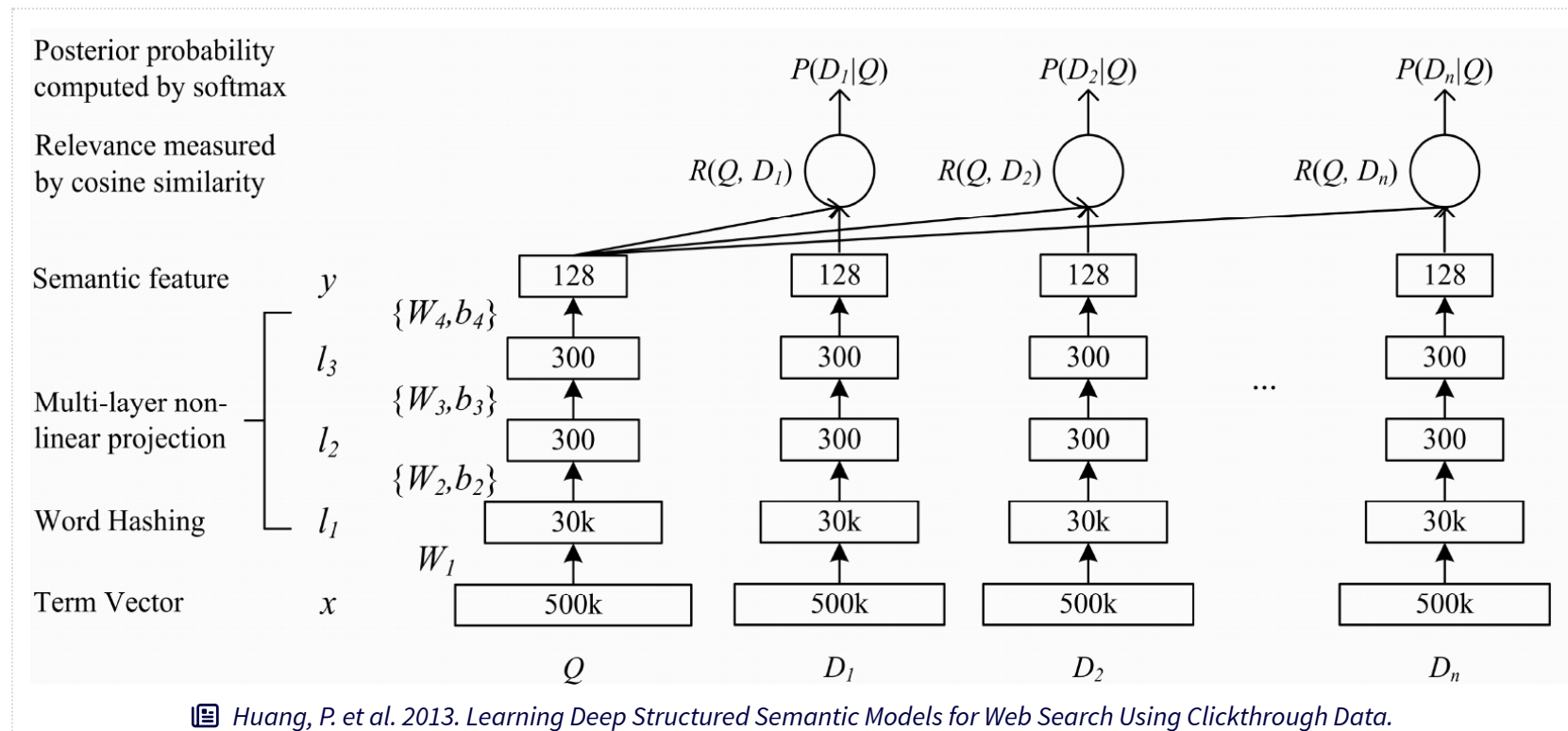
⚠️ Challenge

- Text is a sequence (words, characters)
- Neural networks expect fixed-size inputs
- How to get fixed-size representations?

ONE OF THE FIRST NEURAL IR MODELS



ONE OF THE FIRST NEURAL IR MODELS



There were quite a few works (skipped here) before modern dense architectures based on Transformers (next slide)

☞ All based on the same pattern

A vector space model using transformer representations:

$$\hat{q} = \text{BERT}_{[\text{CLS}]}([\text{CLS}] \ q_1 \dots q_n \ [\text{SEP}])$$

and

$$\hat{d} = \text{BERT}_{[\text{CLS}]}([\text{CLS}] \ d_1 \dots d_m \ [\text{SEP}])$$

The score is given by $s(q, d) = \hat{q} \cdot \hat{d}$

Alternative: use mean pooling instead of [CLS] token

☞ All based on the same pattern

A vector space model using transformer representations:

$$\hat{q} = \text{BERT}_{[\text{CLS}]}([\text{CLS}] \ q_1 \dots q_n \ [\text{SEP}])$$

and

$$\hat{d} = \text{BERT}_{[\text{CLS}]}([\text{CLS}] \ d_1 \dots d_m \ [\text{SEP}])$$

The score is given by $s(q, d) = \hat{q} \cdot \hat{d}$

Alternative: use mean pooling instead of [CLS] token

And... that's it!

NEURAL INFORMATION RETRIEVAL

DENSE INDEXING

⚠ Searching in Dense Space

Computing dot products with all documents is expensive ($O(n)$)

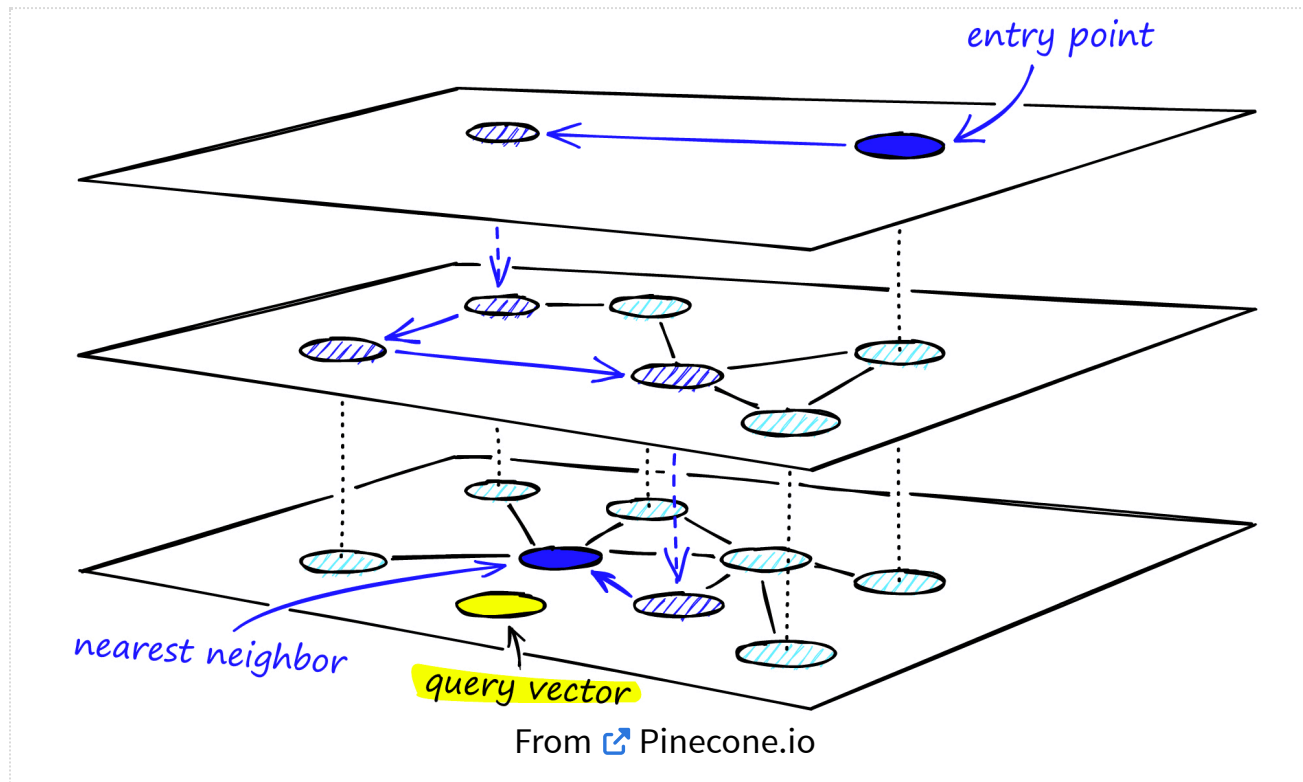
How can we speed up retrieval?

TWO COMPLEMENTARY APPROACHES

- **Graph-based methods** (HNSW): reduce number of comparisons
- **Quantization** (PQ): make each comparison faster

NEURAL INFORMATION RETRIEVAL ► HNSW: HIERARCHICAL NAVIGABLE SMALL WORLD

📖 Malkov, Y.A. and Yashunin, D.A. 2018. *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*.



PROPERTIES

- Search complexity: $O(\log n)$ instead of $O(n)$
- Approximate: may miss some relevant documents
- Trade-off: more layers/neighbors $\rightarrow$ better recall, slower search

NEURAL INFORMATION RETRIEVAL ► PRODUCT QUANTIZATION (PQ)

 Jegou, H. et al. 2011. Product Quantization for Nearest Neighbor Search.

IDEA

- Split d -dimensional vector into m subvectors of dimension d/m
- Quantize each subvector independently using a codebook of k centroids
- Store only the centroid indices (e.g., $m \times 8$ bits instead of $d \times 32$ bits)

EXAMPLE ($d = 128, m = 8, k = 256$)

ORIGINAL VECTOR

128 floats = 512 bytes

PQ ENCODING

8 subvectors $\times$ 1 byte = 8 bytes

COMPRESSION

64× smaller!

DISTANCE COMPUTATION

- Precompute distances from query subvectors to all centroids
- Approximate distance = sum of table lookups

$$d(q, x) \approx \sum_{i=1}^m d(q^{(i)}, c_{\hat{x}_i}^{(i)})$$

FAISS

Facebook AI Similarity Search - library for efficient similarity search

[🔗](#) Official guidelines to choose an index

INDEX TYPES

Flat

Exact search, no compression

IVF (Inverted File)

Cluster-based, probe nearest clusters

HNSW

Hierarchical Navigable Small World graphs

PQ (Product Quantization)

Compress vectors into codes

NEURAL INFORMATION RETRIEVAL

LEARNED SPARSE RETRIEVAL

⚙ Inverted indices are efficient!

- 🔑 Why not use sparse representations?
- 🔑 We can then rank **all documents** efficiently

Idea #1: learn term weights

- 🔑 Idea: Predict term weights using contextualized representations
- 🔑 Based on BERT contextual embeddings + linear model

Table 5: Visualization of DeepCT-Index passage term weights. Red shades reflect the normalized term weights – the percentage of total passage term weights applied to the term. White indicates zero weight. Query terms are bold.

	Percentage of weights a term takes in the passage: 0 10% 20% 30% 40% >50%
Query	who is susan boyle
On-Topic	Amateur vocalist Susan Boyle became an overnight sensation after appearing on the first round of 2009's popular U.K. reality show Britain's Got Talent.
Off-Topic	Best Answer: a troll is generally someone who tries to get attention by posting things everyone will disagree, like going to a susan boyle fan page and writing susan boyle is ugly on the wall. they are usually 14-16 year olds who crave attention.
Query	what values do zoos serve
On-Topic	Zoos serve several purposes depending on who you ask. 1) Park/Garden: Some zoos are similar to a botanical garden or city park. They give people living in crowded, noisy cities a place to walk through a beautiful, well maintained outdoor area. The animal exhibits create interesting scenery and make for a fun excursion.
Off-topic	There are NO purebred Bengal tigers in the U.S. The only purebred tigers in the U.S. are in AZA zoos and include 133 Amur (AKA Siberian), 73 Sumatran and 50 Malayan tigers in the Species Survival Plan. All other U.S. captive tigers are inbred and cross bred and do not serve any conservation value .
Query	do atoms make up dna
On-Topic	DNA only has 5 different atoms - carbon, hydrogen, oxygen, nitrogen and phosphorous. According to one estimation, there are about 204 billion atoms in each DNA .
Off-Topic	Genomics in Theory and Practice. What is Genomics . Genomics is a study of the genomes of organisms. It main task is to determine the entire sequence of DNA or the composition of the atoms that make up the DNA and the chemical bonds between the DNA atoms .

📖 Dai, Z. and Callan, J. 2019. Context-Aware Sentence/Passage Term Importance Estimation For First Stage Retrieval.

☞ Term weight estimated by linear transformation of BERT contextual representation $x^{(t)}$

$$y_{t,d} = x^{(t)} \cdot w_1 + b_1$$

$$y_{t,q} = x^{(t)} \cdot w_2 + b_2$$

Score: $rsv(q, d) = \sum_{t \in q} y_{t,q} \cdot y_{t,d}$

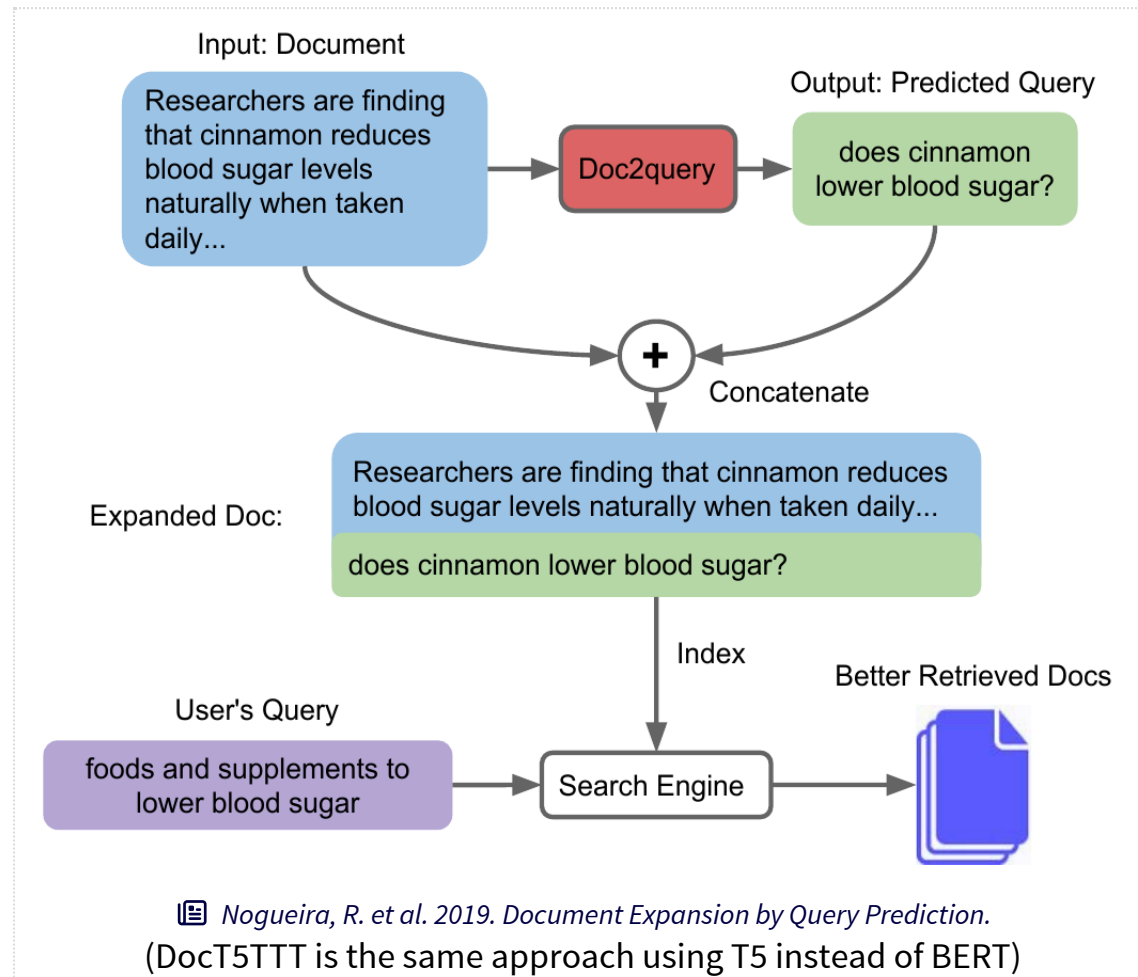
☞ Target values y from relevance estimates on training data

⚡ Absent words are not added (no expansion)

Idea #2: Expand documents with generative models

NEURAL INFORMATION RETRIEVAL ► DOC2QUERY AND DOCT5TTT

🔑 Generate questions that the document answers



💡 Combine vocabulary expansion and term weighting

DocT5 (vocabulary expansion) + DeepCT (term weights)

Table 2: Effectiveness metrics and mean response time (MRT, in ms) for first-stage methods, on MSMARCO Dev Queries, TREC 2019 queries, and TREC 2020 queries. The symbol ∇ denotes a significant difference viz. DeepImpact

Strategy	NDCG@10	MRR@10	MAP	MRT
MSMARCO Dev Queries				
BM25	0.235 [∇]	0.188 [∇]	0.196 [∇]	13.24
DeepCT	0.298 [∇]	0.244 [∇]	0.252 [∇]	10.91
DocT5Query	0.338 [∇]	0.278 [∇]	0.286 [∇]	12.62
DeepImpact	0.385	0.326	0.332	58.64
TREC 2019				
BM25	0.497 [∇]	0.683	0.290 [∇]	10.27
DeepCT	0.578 [∇]	0.714	0.329 [∇]	11.02
DocT5Query	0.648	0.799	0.405	11.76
DeepImpact	0.695	0.863	0.456	51.23
TREC 2020				
BM25	0.483 [∇]	0.659 [∇]	0.286 [∇]	14.67
DeepCT	0.550 [∇]	0.705	0.349 [∇]	12.00
DocT5Query	0.619	0.742	0.408	15.51
DeepImpact	0.651	0.820	0.426	58.00

Table 3: Effectiveness metrics and mean response time (MRT, in ms) using several re-ranking techniques on MSMARCO Dev Queries, TREC 2019 queries, and TREC 2020 queries. ∇ denotes a significant difference viz. ColBERT E2E

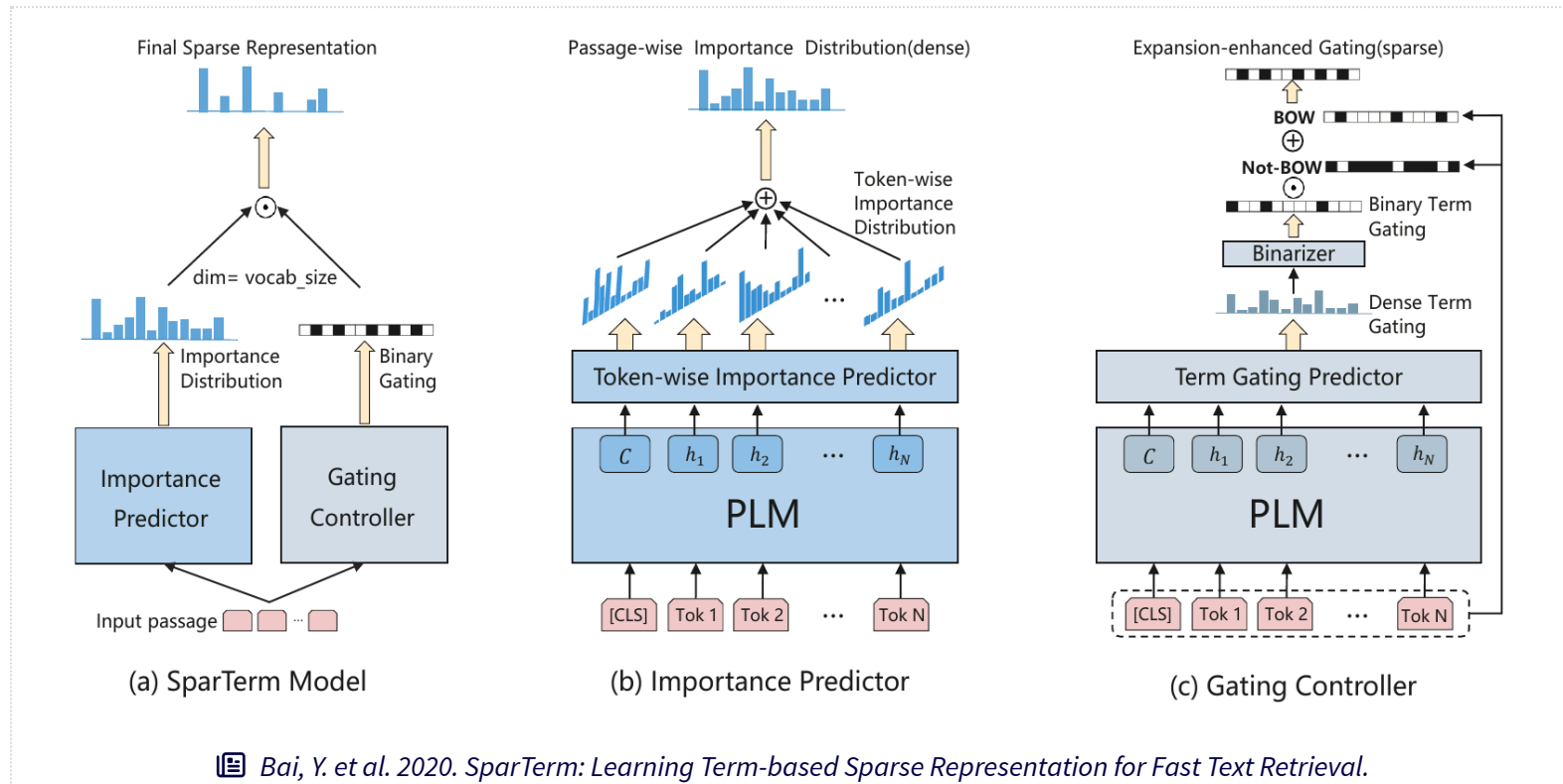
Strategy	NDCG@10	MRR@10	MRT
MSMARCO Dev Queries			
DeepImpact + EPIC	0.367 [∇]	0.303 [∇]	194.64
DeepImpact + ColBERT	0.425	0.362	81.00
ColBERT E2E	0.424	0.361	380.97
TREC 2019			
DeepImpact + EPIC	0.711	0.880	191.23
DeepImpact + ColBERT	0.722	0.826	73.29
ColBERT E2E	0.694	0.826	370.98
TREC 2020			
DeepImpact + EPIC	0.646	0.773	196.00
DeepImpact + ColBERT	0.691	0.781	79.84
ColBERT E2E	0.676	0.776	364.82

📖 Mallia, A. et al. 2021. *Learning Passage Impacts for Inverted Indexes*.

Idea #3: Learn sparse representations directly

NEURAL INFORMATION RETRIEVAL ► SPARTERM AND SPLADE

- 🔑 Predict vocabulary weights for each token
- 🔑 Pooling mechanism (= 1 weight per word at the end)



🔧 MLM task naturally induces vocabulary expansion

>> <code>distilbert-base-uncased</code> [Sanh et al. 2019]	
<i>prostate</i>	→ (<i>prostate</i> , <i>.</i> , <i>the</i>)
<i>cancer</i>	→ (<i>cancer</i> , <i>tumor</i> , <i>##mour</i>)
<i>detection</i>	→ (<i>detection</i> , <i>identification</i> , <i>screening</i>)
<i>treatment</i>	→ (<i>.</i> , <i>;</i> , <i>treatment</i>)
>> <code>google/electra-base-generator</code> [Clark et al. 2020]	
<i>prostate</i>	→ (<i>prostate</i> , <i>##iate</i> , <i>mal</i>)
<i>cancer</i>	→ (<i>cancer</i> , <i>cancers</i> , <i>tumor</i>)
<i>detection</i>	→ (<i>detection</i> , <i>detecting</i> , <i>screening</i>)
<i>treatment</i>	→ (<i>treatment</i> , <i>therapy</i> , <i>treatments</i>)
>> <code>naver/splade-cocondenser-ensembledistil</code> [Formal et al. 2021c]	
<i>prostate</i>	→ (<i>prostate</i> , <i>bp</i> , <i>ur</i>)
<i>cancer</i>	→ (<i>cancer</i> , <i>tumor</i> , <i>disease</i>)
<i>detection</i>	→ (<i>detection</i> , <i>detect</i> , <i>test</i>)
<i>treatment</i>	→ (<i>treatment</i> , <i>therapy</i> , <i>treated</i>)

Table 3.1: MLM prediction for the Robust04 query “*prostate cancer detection treatment*”. For each token, we show the top-3 predicted tokens, i.e., with the highest MLM logits. Note that these models use the WordPiece vocabulary (Section 2.4.2). Also, note that only the generator part of ELECTRA has been pre-trained with MLM. We also include for comparison the prediction for one of our SPLADE model.

Formal, T. et al. 2021. SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval.

Estimate weight w_{td} of term t in document d :

$$w_{td} = \log \left(1 + \max_{i \in d} \text{ReLU}(y_i^{(t)}) \right)$$

with

$$y_i^{(t)} = \text{BERT}_i(d) \cdot \theta_t + b_t$$

where $\text{BERT}_i(d)$ is the contextualized embedding of token i in document d

TRAINING OBJECTIVE

- **IR Loss:** infoNCE or distillation
- **Sparsity Loss:** FLOPS regularization to limit number of active terms

$$\mathcal{L}_{\text{FLOPS}} = \lambda_d \sum_{t \in V} \bar{w}_{t,d}^2 + \lambda_q \sum_{t \in V} \bar{w}_{t,q}^2$$

where $\bar{w}_{t,q}$ and $\bar{w}_{t,d}$ are mean weights of term t over queries / documents (batch)

- ✚ Excellent results (much better than BM25)
- ✚ Easy to train (SparTerm, SPLADE) or use (DocT5TTT)
- ✖ To reach state-of-the-art, still need re-ranking (e.g., with monoBERT)

NEURAL INFORMATION RETRIEVAL

SPARSE INDEXING

CLASSIC APPROACH

- Map each term to list of documents containing it
- Store term frequencies and positions
- Efficient for exact term matching
- Well-understood compression techniques

FOR LEARNED SPARSE (SPLADE, ETC.)

- Same structure, but with learned term weights
- May have expanded vocabulary
- Can use standard search engines (Lucene, Elasticsearch)

 Broder, A.Z. et al. 2003. *Efficient Query Evaluation Using a Two-Level Retrieval Process*. *Proceedings of the Twelfth International Conference on Information and Knowledge Management*.

BASIC IDEA

- Weighted **AND** - efficient top-k retrieval
- Each posting list has a **max weight** (upper bound for any doc in the list)
- Find **pivot**: smallest doc_id where cumulative max weights $\geq$ threshold

HOW IT WORKS

1. Accumulate max weights until $\text{sum} \geq \theta \rightarrow$ this term's doc_id is the **pivot**
2. If all cursors point to pivot $\rightarrow$ **evaluate** document, update θ
3. Otherwise $\rightarrow$ **skip**: advance cursors with doc_id < pivot to pivot

 Key: we can jump directly to pivot, skipping many documents

SETUP

- Query: "cat food", top-1
- $\theta = 0$ initially

POSTING LISTS

Term	Max	Postings (doc: weight)
cat	3.0	d1: 2.5, d2: 3.0, d3, d4, d5: 3.0
food	4.0	d2: 3.0, d5: 2.0

Evaluated / Skipped

WAND EXECUTION

1. cat→d1, food→d2

Accumulate: $\text{cat}(3.0) \geq \theta=0 \rightarrow \text{pivot}=\mathbf{d1}$

Only cat at d1 → **evaluate**: score=2.5, $\theta=2.5$

Advance cat→d2

2. cat→d2, food→d2

Accumulate: $\text{cat}(3.0) \geq \theta \rightarrow \text{pivot}=\mathbf{d2}$

Both at d2 → **evaluate**: score=6.0, $\theta=6.0$

Advance cat→d3, food→d5

3. cat→d3, food→d5

Accumulate: $\text{cat}(3.0) < \theta$, $\text{cat}+\text{food}(7.0) \geq \theta \rightarrow \text{pivot}=\mathbf{d5}$

$\text{cat}(d3) < \text{pivot} \rightarrow$ **skip cat to d5**, both at d5

evaluate: score=5.0 < θ , done

🔊 d3, d4 skipped: cat cursor jumped directly to d5

THE PROBLEM

- Learned weights are floating-point values
- Standard inverted indices use integer term frequencies
- Use quantization to convert learned weights to integers

IN PRACTICE

🔧 Simple solution: repeat tokens according to quantized weight

SPLADE OUTPUT

```
{"cat": 3.7, "food": 2.1, "pet": 1.8}
```

QUANTIZATION (10 STEPS, MAX=5)

```
cat: round(3.7/5 × 10) = 7  
food: round(2.1/5 × 10) = 4  
pet: round(1.8/5 × 10) = 4
```

PYTERRIER INPUT

```
"cat cat cat cat cat cat cat  
food food food food  
pet pet pet pet"
```

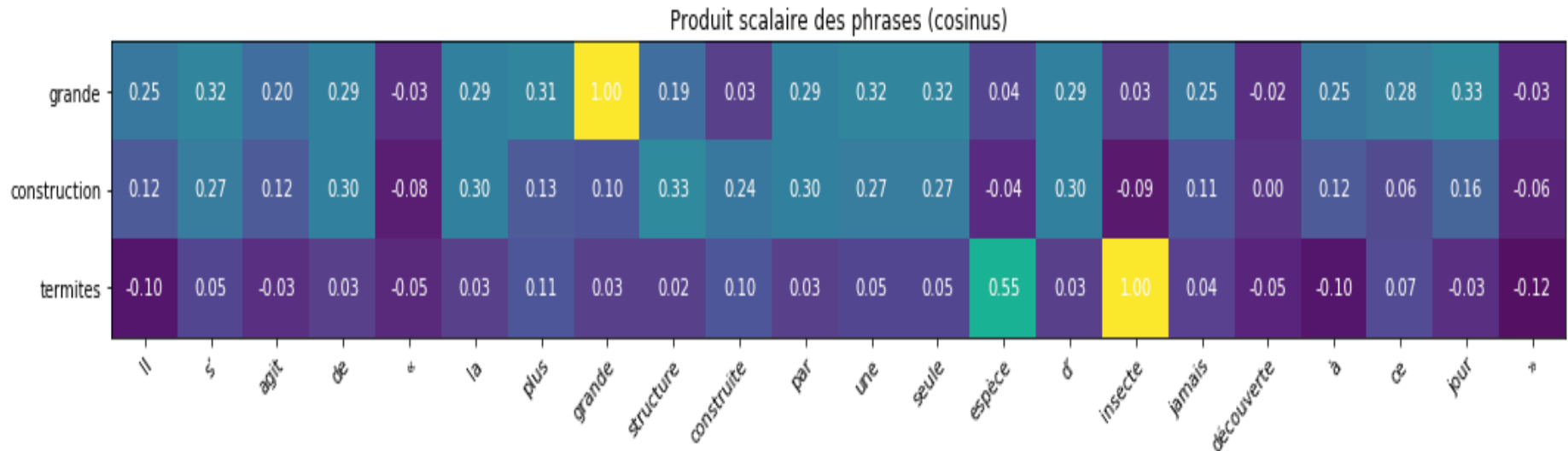
NEURAL INFORMATION RETRIEVAL INTERACTION MODELS

NEURAL INFORMATION RETRIEVAL ► MOTIVATION

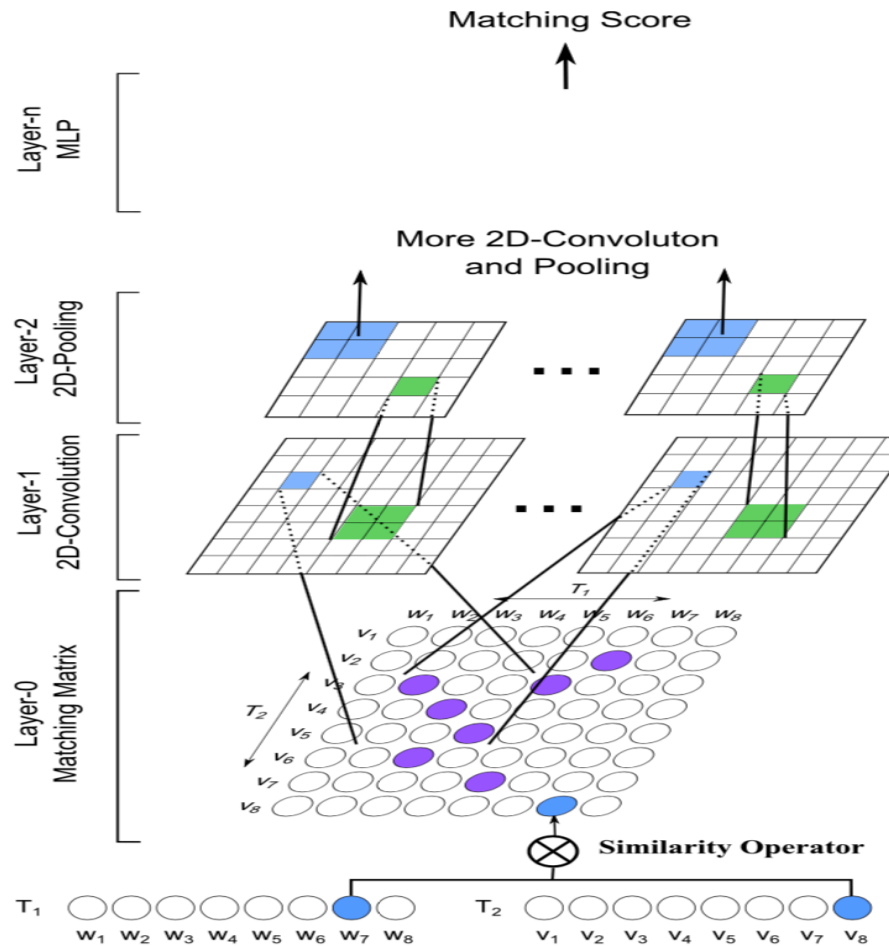
Build attention maps from both texts

Benefit: Identify common terms (exact match) and similar terms (semantic match)

Use 2D aggregation techniques (convolution and/or pooling)



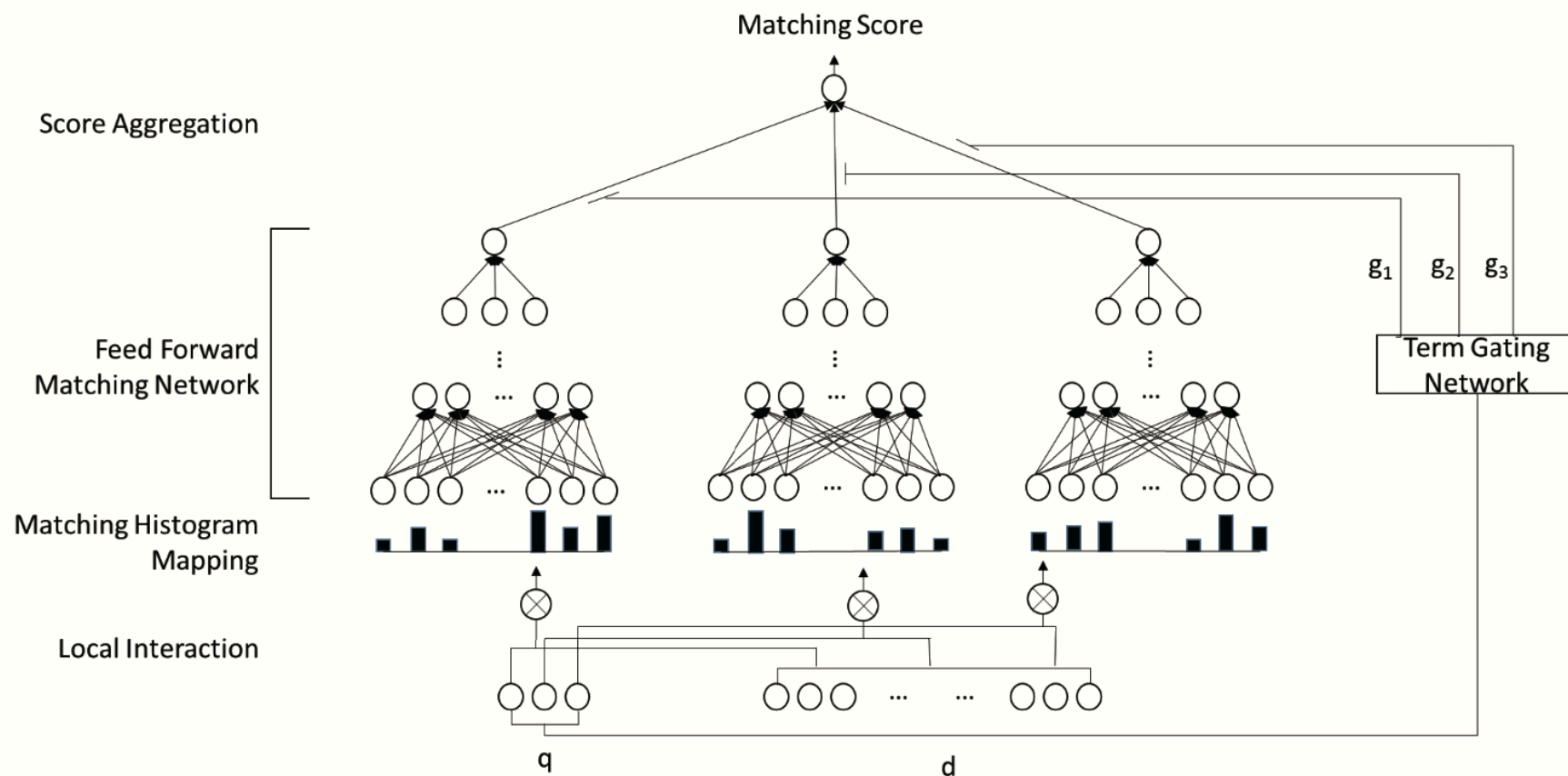
NEURAL INFORMATION RETRIEVAL ► MATCHPYRAMID



💡 Use object detection techniques

📖 Hu, B. et al. 2014. Convolutional Neural Network Architectures for Matching Natural Language Sentences. *Advances in Neural Information Processing Systems* 27.

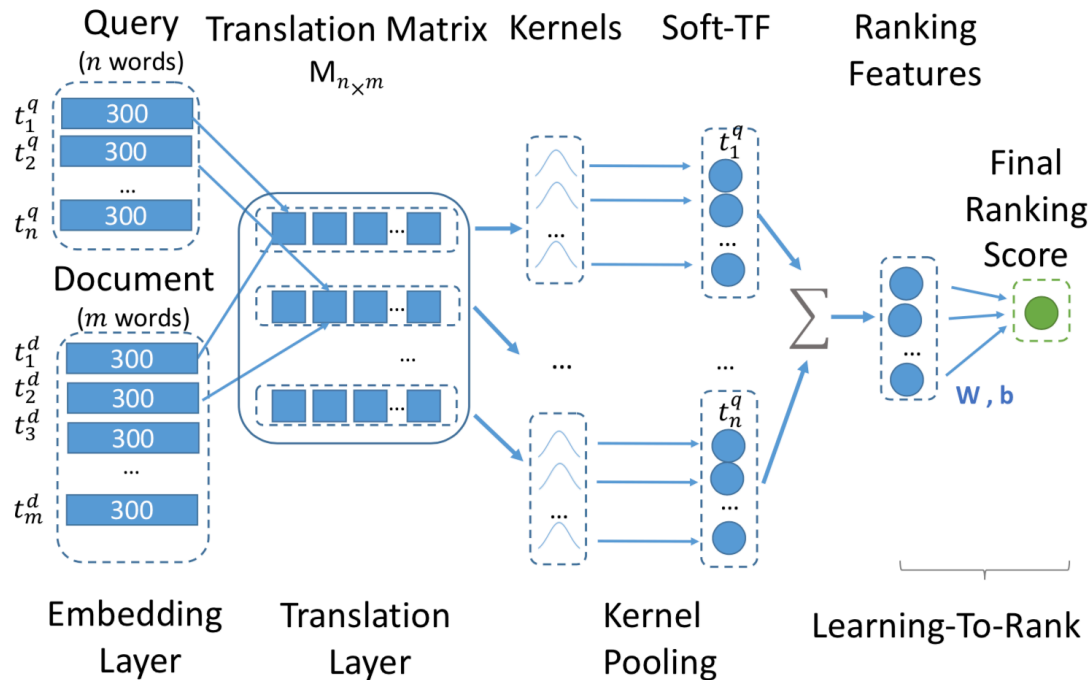
- Interaction matrix
- 2D convolution model



 Guo, J. et al. 2016. A Deep Relevance Matching Model for Ad-hoc Retrieval.

NEURAL INFORMATION RETRIEVAL ► KNRM: KERNEL NEURAL RELEVANCE MODEL

🏆 One of the best interaction models (before BERT...)



Generalization of DRMM

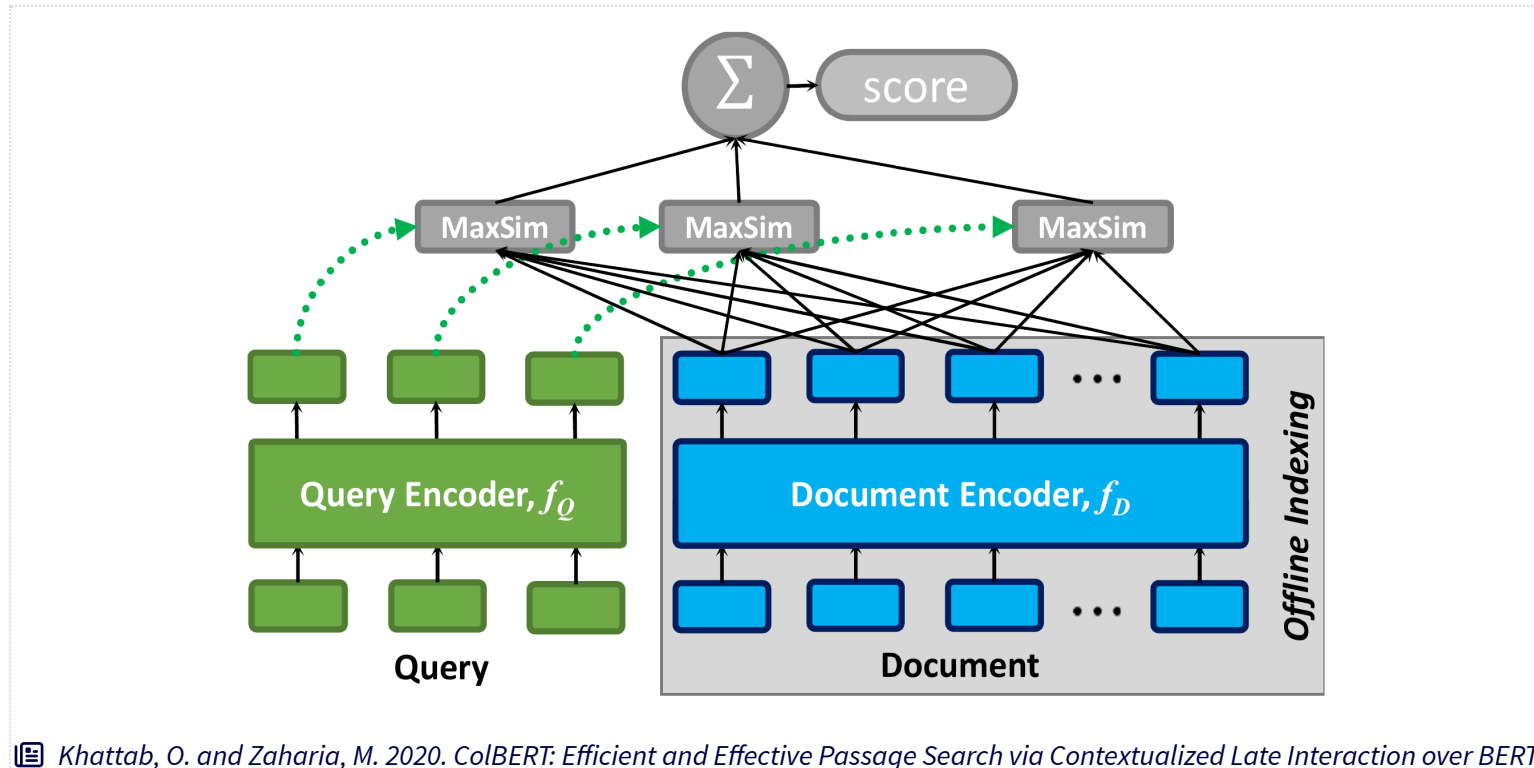
$$K_k(\underbrace{q_i \cdot d_j}_{I_{ij}}) = \exp \left(\frac{(I_{ij} - \mu_k)^2}{2\sigma_k^2} \right)$$

with q_i the representation of query term i and d_j of document term j

📖 Xiong, C. et al. 2017. *End-to-End Neural Ad-hoc Ranking with Kernel Pooling*.

NEURAL INFORMATION RETRIEVAL ► COLBERT

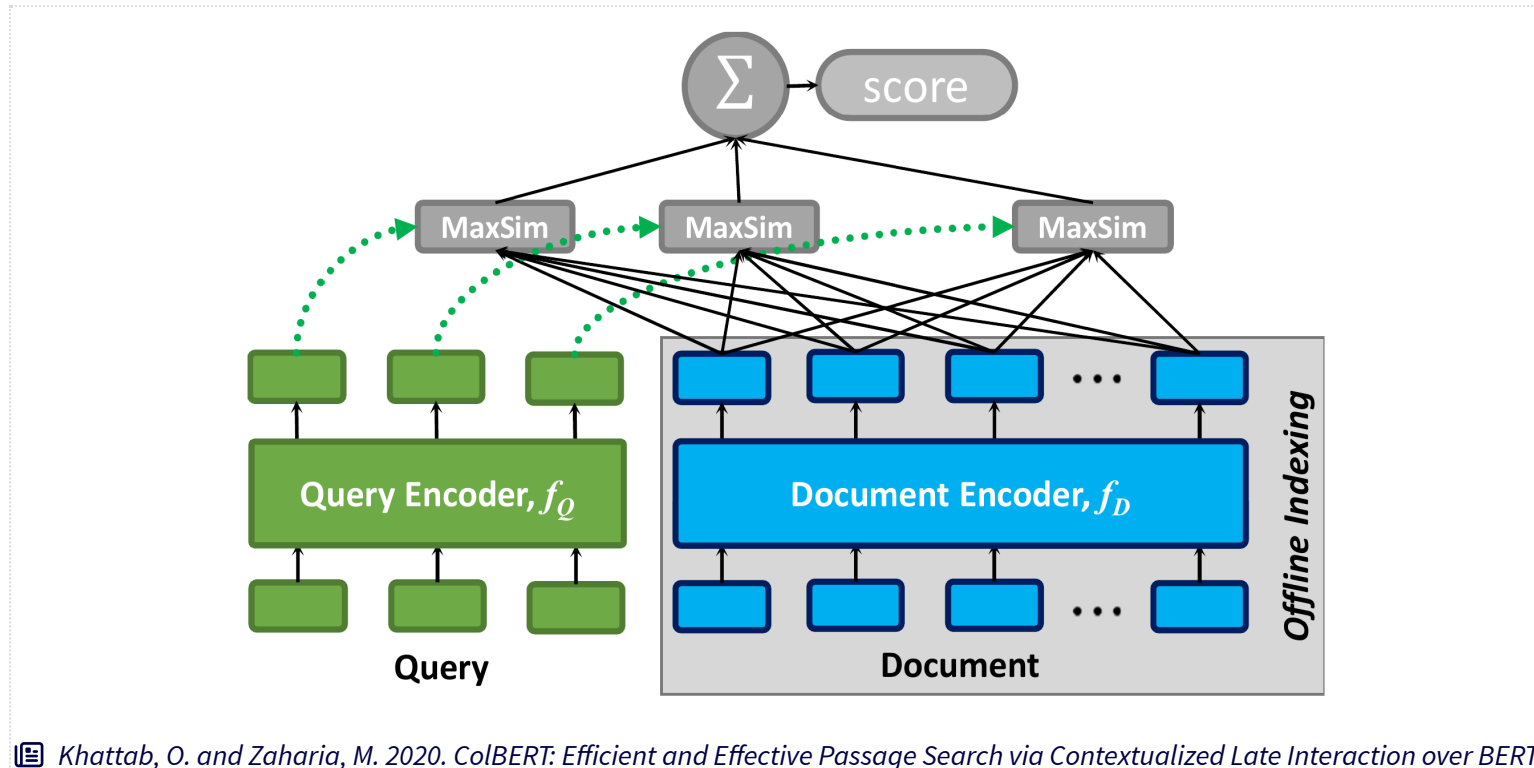
- 🔑 Even simpler: Interaction model with max aggregation
- 🔑 **Key advantage:** Can build an index!



$$score(q, d) = \sum_{i=1}^{|q|} \max_{j=1}^{|d|} \underbrace{sim(q_i, d_j)}$$

NEURAL INFORMATION RETRIEVAL ► COLBERT

- Even simpler: Interaction model with max aggregation
- Key advantage:** Can build an index!



$$score(q, d) = \sum_{i=1}^{|q|} \max_{j=1}^{|d|} \underbrace{sim(q_i, d_j)}_{\text{cosine or inner product}}$$

ADVANTAGES

-  Better effectiveness than bi-encoders
Fine-grained token-level matching
-  Can build an ANN index
Pre-compute document token embeddings offline
-  Faster than cross-encoders
No query-document attention at query time

DISADVANTAGES

-  Storage: store all token embeddings
~128 dims × ~180 tokens × 4 bytes ≈ 90 KB/doc
MS MARCO (8.8M docs) ≈ 800 GB
-  Speed: ANN search per query token
 $|q|$ ANN lookups instead of 1

Method	MRR@10 (Dev)	MRR@10 (Local Eval)	Latency (ms)	Recall@50	Recall@200	Recall@1000
BM25 (official)	16.7	-	-	-	-	81.4
BM25 (Anserini)	18.7	19.5	62	59.2	73.8	85.7
doc2query	21.5	22.8	85	64.4	77.9	89.1
DeepCT	24.3	-	62 (<i>est.</i>)	69 [3]	82 [3]	91 [3]
docTTTTTquery	27.7	28.4	87	75.6	86.9	94.7
ColBERT _{L2} (re-rank)	34.8	36.4	-	75.3	80.5	81.4
ColBERT _{L2} (end-to-end)	36.0	36.7	479	82.9	92.3	96.8

🔧 Compression technique

$$\hat{x}_{di} = \underbrace{z_{j(x_{di})}}_{\text{cluster}} + \underbrace{\tilde{x}_{di}}_{\text{residual}}$$

📖 Santhanam, K. et al. 2021. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction.

🔧 PLAID: 3 increasingly precise filtering stages, then full computation

System	MRR@10	R@100	R@1k	Latency (ms)		
				1-CPU	8-CPU	GPU
BM25 (PISA [34]; $k = 1000$)	18.7*	-	-	8.3*	-	-
SPLADEv2 (PISA; $k = 1000$)	36.8*	-	97.9*	220.3*	-	-
ColBERTv1	36.1	87.3	95.2	-	-	54.3
Vanilla ColBERTv2 ($p=2, c=2^{13}$)	39.7	90.4	96.6	3485.1	921.8	53.4
Vanilla ColBERTv2 ($p=4, c=2^{16}$)	39.7	91.4	98.3	-	4568.5	259.6
PLAID ColBERTv2 ($k = 10$)	39.4	-	-	185.5	31.5	11.5
PLAID ColBERTv2 ($k = 100$)	39.8	90.6	-	222.3	52.9	20.2
PLAID ColBERTv2 ($k = 1000$)	39.8	91.3	97.5	352.3	101.3	38.4

Table 3: End-to-end in-domain evaluation on the MS MARCO v1 benchmark. Numbers marked with an asterisk are copied from Formal et al. [11] for SPLADEv2 quality and Mackenzie et al. [30] for latencies.

📖 Santhanam, K. et al. 2022. PLAID: An Efficient Engine for Late Interaction Retrieval.

System	Success@5	Success@100	Latency (ms)	
			CPU (8)	GPU
BM25	47.8*	77.6*	-	-
SPLADEv2	67.0*	89.0*	-	-
Vanilla ColBERTv2 ($p=2, c=2^{13}$)	69.3	90.3	1508.4	66.9
PLAID ColBERTv2 ($k = 10$)	69.1	-	35.5	9.2
PLAID ColBERTv2 ($k = 100$)	69.4	89.9	64.8	17.4
PLAID ColBERTv2 ($k = 1000$)	69.6	90.5	163.1	27.3

Table 5: End-to-end out-of-domain evaluation on the (dev) pooled dataset of the LoTTE benchmark. Numbers marked with an asterisk were taken from Santhanam et al. [42].

 Interaction models achieve the best performance

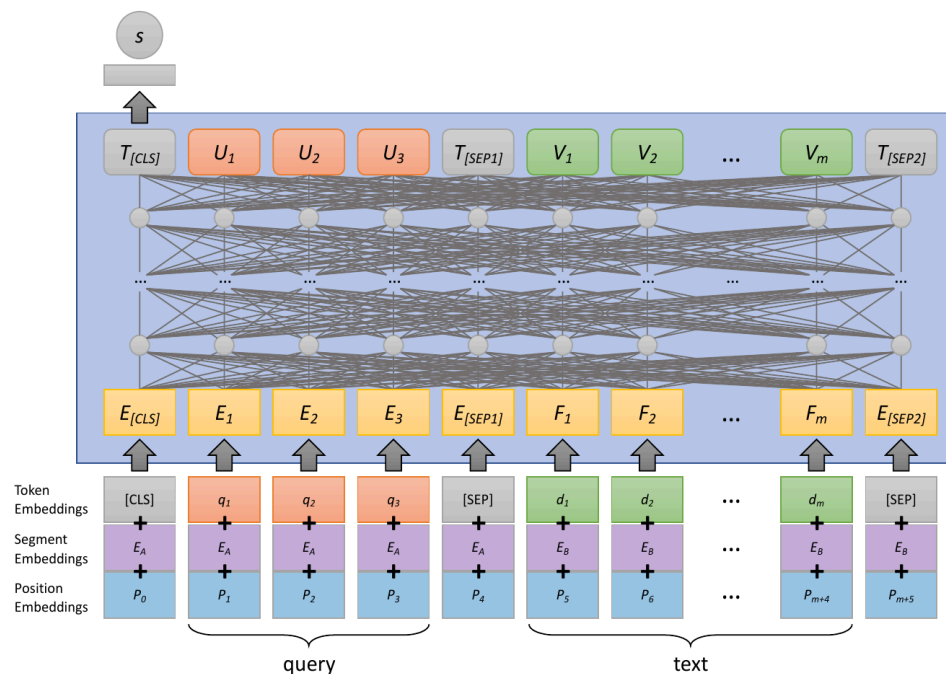
 Computationally expensive - though recent work shows significant simplification is possible while maintaining quality

 (Generally, except ColBERT) Two-stage mechanism: select ~1000 documents then re-rank

NEURAL INFORMATION RETRIEVAL

CROSS-ENCODERS

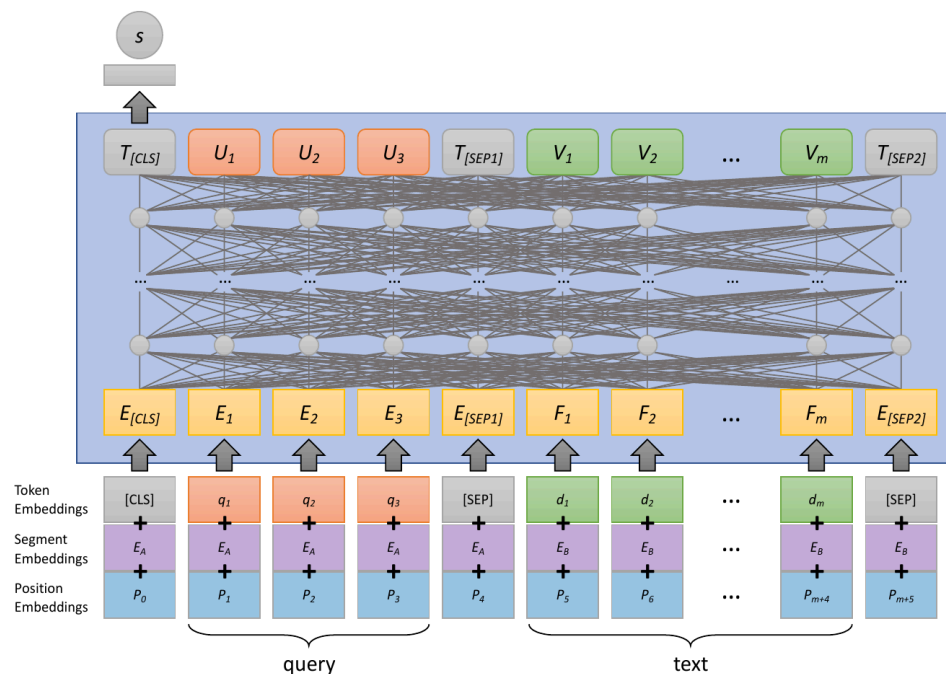
🔧 Concatenate query and target document, feed to BERT



📖 Nogueira, R. and Cho, K. 2019. *Passage Re-ranking with BERT*.

Score $rsv(q, d)$ is computed from the [CLS] token representation (2-layer MLP)

🔑 Concatenate query and target document, feed to BERT



📖 Nogueira, R. and Cho, K. 2019. *Passage Re-ranking with BERT*.

Score $rsv(q, d)$ is computed from the $[CLS]$ token representation (2-layer MLP)

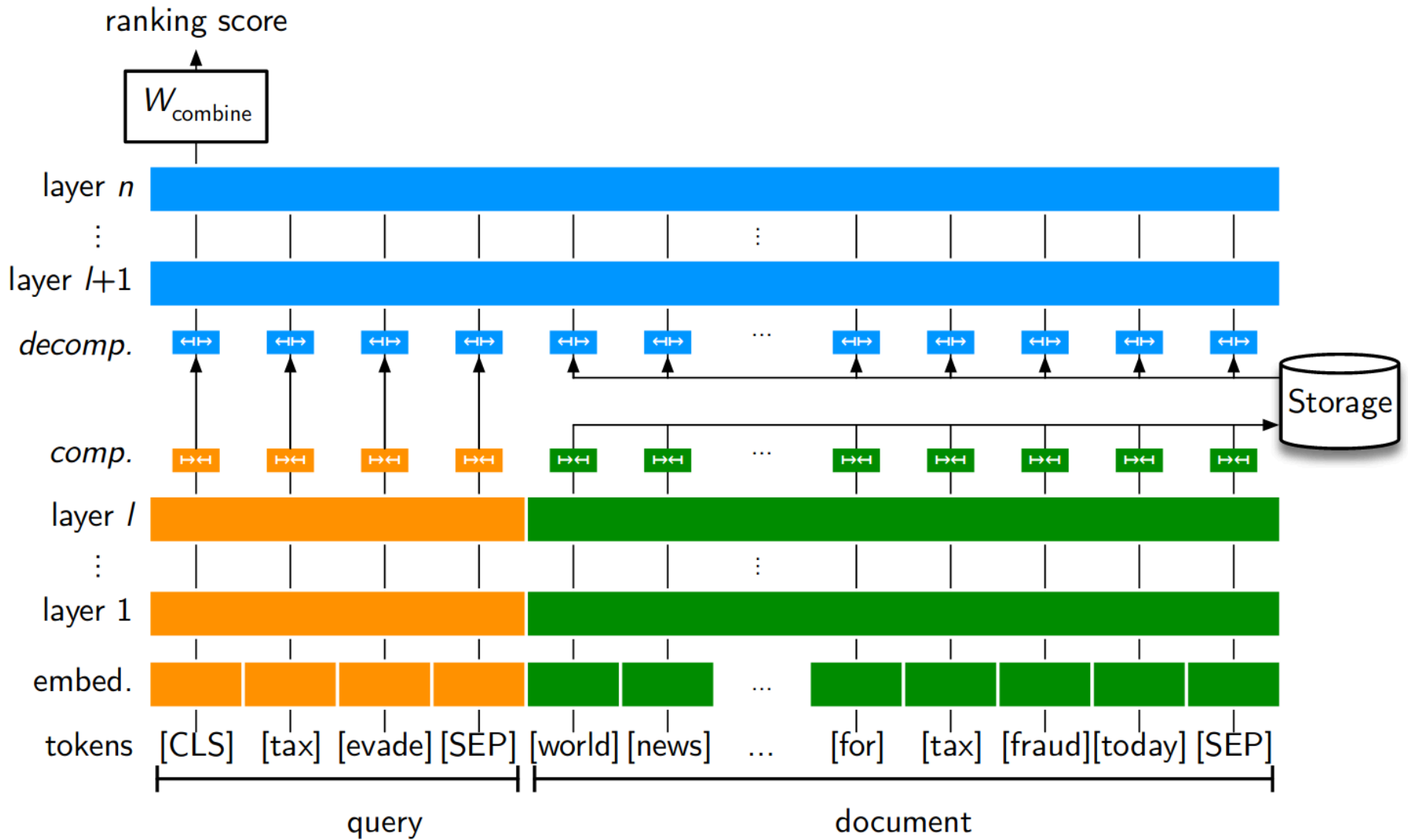
And... that's it!

Method		TREC 2019 DL Passage		
		nDCG@10	MAP	Recall@1k
(3a)	BM25 (Anserini, $k = 1000$)	0.5058	0.3773	0.7389
(3b)	+ monoBERT _{Large}	0.7383	0.5058	0.7389
(4a)	BM25 + RM3 (Anserini, $k = 1000$)	0.5180	0.4270	0.7882
(4b)	+ monoBERT _{Large}	0.7421	0.5291	0.7882

✚ Excellent results...

✖ But very expensive: recent work focuses on reducing processing time substantially

NEURAL INFORMATION RETRIEVAL ► LATE INTERACTION: PRETTR



Ranker	TREC WebTrack 2012					Robust04
	Total	Speedup	Query	Decom.	Combine	Total
Base	1.941s	(1.0×)	-	-	-	2.437s
$l = 1$	1.768s	(1.1×)	2ms	10ms	1.756s	2.222s
$l = 2$	1.598s	(1.2×)	3ms	10ms	1.585s	2.008s
$l = 3$	1.423s	(1.4×)	5ms	10ms	1.409s	1.792s
$l = 4$	1.253s	(1.5×)	6ms	10ms	1.238s	1.575s
$l = 5$	1.080s	(1.8×)	7ms	10ms	1.063s	1.356s
$l = 6$	0.906s	(2.1×)	9ms	10ms	0.887s	1.138s
$l = 7$	0.735s	(2.6×)	10ms	10ms	0.715s	0.922s
$l = 8$	0.562s	(3.5×)	11ms	10ms	0.541s	0.704s
$l = 9$	0.391s	(5.0×)	12ms	10ms	0.368s	0.479s
$l = 10$	0.218s	(8.9×)	14ms	10ms	0.194s	0.266s
$l = 11$	0.046s	(42.2×)	15ms	10ms	0.021s	0.053s

MacAvaney, S. et al. 2020. Efficient Document Re-Ranking for Transformers by Precomputing Term Representations. *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*.

LEARNING TO RANK

LEARNING TO RANK

PROBLEM FORMULATION

LEARNING TO RANK ► THE LEARNING TO RANK PROBLEM

☞ Over time, many relevance indicators have been defined...

How to combine them ?

☞ Each model (BM25, PageRank) measures a form of relevance

☞ How to combine them optimally?

ID	Feature Description	Category
1	$\sum_{q_i \in q \cap d} c(q_i, d)$ in body	Q-D
2	$\sum_{q_i \in q \cap d} c(q_i, d)$ in anchor	Q-D
3	$\sum_{q_i \in q \cap d} c(q_i, d)$ in title	Q-D
4	$\sum_{q_i \in q \cap d} c(q_i, d)$ in URL	Q-D
5	$\sum_{q_i \in q \cap d} c(q_i, d)$ in whole document	Q-D
6	$\sum_{q_i \in q} idf(q_i)$ in body	Q
7	$\sum_{q_i \in q} idf(q_i)$ in anchor	Q
8	$\sum_{q_i \in q} idf(q_i)$ in title	Q
9	$\sum_{q_i \in q} idf(q_i)$ in URL	Q
10	$\sum_{q_i \in q} idf(q_i)$ in whole document	Q
11	$\sum_{q_i \in q \cap d} c(q_i, d) \cdot idf(q_i)$ in body	Q-D
12	$\sum_{q_i \in q \cap d} c(q_i, d) \cdot idf(q_i)$ in anchor	Q-D
13	$\sum_{q_i \in q \cap d} c(q_i, d) \cdot idf(q_i)$ in title	Q-D
14	$\sum_{q_i \in q \cap d} c(q_i, d) \cdot idf(q_i)$ in URL	Q-D
15	$\sum_{q_i \in q \cap d} c(q_i, d) \cdot idf(q_i)$ in whole document	Q-D
16	$ d $ of body	D
17	$ d $ of anchor	D
18	$ d $ of title	D
19	$ d $ of URL	D
20	$ d $ of whole document	D

21	BM25 of body	Q-D
22	BM25 of anchor	Q-D
23	BM25 of title	Q-D
24	BM25 of URL	Q-D
25	BM25 of whole document	Q-D
26	LMIR.ABS of body	Q-D
27	LMIR.ABS of anchor	Q-D
28	LMIR.ABS of title	Q-D
29	LMIR.ABS of URL	Q-D
30	LMIR.ABS of whole document	Q-D
31	LMIR.DIR of body	Q-D
32	LMIR.DIR of anchor	Q-D
33	LMIR.DIR of title	Q-D
34	LMIR.DIR of URL	Q-D
35	LMIR.DIR of whole document	Q-D
36	LMIR.JM of body	Q-D
37	LMIR.JM of anchor	Q-D
38	LMIR.JM of title	Q-D
39	LMIR.JM of URL	Q-D
40	LMIR.JM of whole document	Q-D
41	Sitemap based term propagation	Q-D
42	Sitemap based score propagation	Q-D
43	Hyperlink based score propagation: weighted in-link	Q-D
44	Hyperlink based score propagation: weighted out-link	Q-D
45	Hyperlink based score propagation: uniform out-link	Q-D
46	Hyperlink based propagation: weighted in-link	Q-D
47	Hyperlink based feature propagation: weighted out-link	Q-D
48	Hyperlink based feature propagation: uniform out-link	Q-D
49	HITS authority	Q-D
50	HITS hub	Q-D
51	PageRank	D
52	HostRank	D
53	Topical PageRank	Q-D
54	Topical HITS authority	Q-D
55	Topical HITS hub	Q-D

☞ A scoring function $\varphi(q, d)$

First article

 Fuhr, N. and Buckley, C. 1991. *A Probabilistic Learning Approach for Document Indexing*.

$$\varphi(q, d) = \log P(R|q, d) = 1 - \exp(\Phi(q, d) \cdot \theta)$$

☞ An IR dataset

- Queries q
- Document collection D
- Relevant documents $D_q^+ \subseteq D$
- Non-relevant documents $D_q^- \subseteq D$

☞ We assume documents $D \setminus D_q^+$ are **non-relevant**

 This is not always true!

LEARNING TO RANK

TWO-STAGE RANKING

φ is expensive to compute ?

φ has been learned... but it's slow!

How can we make it faster?

φ is expensive to compute ?

φ has been learned... but it's slow!

How can we make it faster?

💡 Two-Stage Ranking

1. Find top-K (e.g., $K = 1000$) with BM25 or another **fast** model
2. Re-rank the K documents with φ

(1) RE-RANKING

- ☞ Use examples after BM25 (or other first-stage retriever)

(2) FULL RANKING

- ☞ Sample from the entire document collection D

- ☞ In both cases, always sample positive examples for training, but:

Full Ranking

Sample negatives using... other IR models

📖 Hofstätter, S. et al. 2021. *Improving Efficient Neural Ranking Models with Cross-Architecture Knowledge Distillation*.

Re-ranking

Sample documents to re-rank using BM25

LEARNING TO RANK

LOSS FUNCTIONS

How to learn a model that maximizes an IR metric ?

- ☞ Not possible in general (metrics are non-differentiable)
- ☞ Use relaxations of the problem

How to learn a model that maximizes an IR metric ?

- ☞ Not possible in general (metrics are non-differentiable)
- ☞ Use relaxations of the problem

☞ Three approaches:

Pointwise

Predict relevance score for each (query, document) pair independently

Mostly classification (relevant/not relevant)

Pairwise

Learn to order pairs: given (q, d^+, d^-) , predict $s(q, d^+) > s(q, d^-)$

Binary classification on document pairs

Listwise

Optimize directly over the full ranked list

Approximate non-differentiable IR metrics (e.g., softmax over scores)

► PCE (Classification)

$$\Delta_{PCE}(p_{\theta}(q, d, y)) = y \log p_{\theta}(q, d) + (1 - y) \log(1 - p_{\theta}(q, d))$$

- ☞ Easy to implement
- ☞ Can be unstable during training

▼ Max-Margin / Triplet Loss

$$\Delta(f_{\theta}(q, d_{+}), f_{\theta}(q, d_{-})) = \max \{0, 1 - (f_{\theta}(q, d_{+}) - f_{\theta}(q, d_{-}))\}$$

Minimum (0) achieved when $f_{\theta}(q, d_{+}) > f_{\theta}(q, d_{-}) + 1$

InfoNCE

Loss defined over a list of documents $\approx$ IR metric

Currently the most widely used in L2R / IR

$$\Delta(f, q, \tilde{D}) = \sum_{d_+ \in \tilde{D} \cap D_q^+} R(q, d_+) \log \frac{\exp(f(q, d_+))}{\sum_{d \in \tilde{D}} \exp(f(q, d))}$$

where R is 1 if d is relevant for q (0 otherwise)

- ☞ Very common for pairwise ($\tilde{D}$ with two documents) re-ranking
- ☞ For ranking, use batch documents as negatives
- ☞ Key challenge: finding the best $\tilde{D}$

📖 Lindgren, E. et al. 2021. *Efficient Training of Retrieval Models Using Negative Cache*.

LEARNING TO RANK ► LOSS: SOFT RANK (LISTWISE)

📖 Taylor, M. et al. 2008. *SoftRank: Optimizing Non-Smooth Rank Metrics*. WSDM.

🔧 Can approximate nDCG with SoftNDCG

$$\mathbb{E} [nDCG] \approx \sum_{q \in Q} \frac{g(d|q)}{CG_{\max}(q)} \sum_{r \geq 1} d(r) \cdot p(\mathbf{R} = r | q, d)$$

LEARNING TO RANK ► LOSS: SOFTRANK (LISTWISE)

📖 Taylor, M. et al. 2008. *SoftRank: Optimizing Non-Smooth Rank Metrics*. WSDM.

🔧 Can approximate nDCG with SoftNDCG

$$\mathbb{E}[nDCG] \approx \sum_{q \in Q} \frac{g(d|q)}{CG_{\max}(q)} \sum_{r \geq 1} d(r) \cdot p(\mathbf{R} = r | q, d)$$

Two questions ?

1. Why $\approx$?

LEARNING TO RANK ► LOSS: SOFTRANK (LISTWISE)

📖 Taylor, M. et al. 2008. *SoftRank: Optimizing Non-Smooth Rank Metrics*. WSDM.

🔧 Can approximate nDCG with SoftNDCG

$$\mathbb{E}[nDCG] \approx \sum_{q \in Q} \frac{g(d|q)}{CG_{\max}(q)} \sum_{r \geq 1} d(r) \cdot p(\mathbf{R} = r|q, d)$$

Two questions ?

1. Why $\approx$?
2. How to estimate $p(\mathbf{R} = r|q, d)$?

LEARNING TO RANK ► LOSS: SOFTRANK (LISTWISE)

📖 Taylor, M. et al. 2008. *SoftRank: Optimizing Non-Smooth Rank Metrics*. WSDM.

🔧 Can approximate nDCG with SoftNDCG

$$\mathbb{E}[nDCG] \approx \sum_{q \in Q} \frac{g(d|q)}{CG_{\max}(q)} \sum_{r \geq 1} d(r) \cdot p(\mathbf{R} = r|q, d)$$

Two questions ?

1. Why $\approx$?
2. How to estimate $p(\mathbf{R} = r|q, d)$?

Use normal distribution to compute $p(\mathbf{R} = r|q, d)$

LEARNING TO RANK ► LOSS: SOFTRANK (LISTWISE)

📖 Taylor, M. et al. 2008. *SoftRank: Optimizing Non-Smooth Rank Metrics. WSDM.*

🔗 Can approximate nDCG with SoftNDCG

$$\mathbb{E}[nDCG] \approx \sum_{q \in Q} \frac{g(d|q)}{CG_{\max}(q)} \sum_{r \geq 1} d(r) \cdot p(\mathbf{R} = r|q, d)$$

Two questions ?

1. Why $\approx$?
2. How to estimate $p(\mathbf{R} = r|q, d)$?

Use normal distribution to compute $p(\mathbf{R} = r|q, d)$

Let π_{ij} be probability that d_i ranks before d_j

Assuming $S_i \sim \mathcal{N}(\mu_i, \sigma^2)$ where μ_i is the retrieval score

🔗 $\pi_{ij} \Leftrightarrow S_i - S_j > 0$ corresponds to CDF of normal distribution (differentiable)

LEARNING TO RANK

TRAINING STRATEGIES

💡 More data but noisier

User actions

- Query reformulations
- Clicks
- Dwell time

💡 Few labeled data but have models

We have queries (logs or generated, e.g., anchor text)

... but no relevance labels

🔑 Use existing models as teachers!

📖 Dehghani, M. et al. 2017. *Learning to Attend, Copy, and Generate for Session-Based Query Suggestion*.



Born Again Neural Networks

A student trained to mimic a teacher can match or exceed the teacher's performance

 Furlanello, T. et al. 2018. *Born-Again Neural Networks*. ICML.



Born Again Neural Networks

A student trained to mimic a teacher can match or exceed the teacher's performance

 Furlanello, T. et al. 2018. *Born-Again Neural Networks*. ICML.

WHY DISTILLATION IN IR?

1. **Expensive teachers:** Cross-encoder models are accurate but too slow for retrieval
2. **False negatives:** Collections have few labels, many relevant docs marked as non-relevant
3. **No labels:** We have queries (logs, generated) but no relevance judgments

THE IDEA

- 📖 Ding, Y.Q.Y. et al. 2020. *RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering*. NAACL.
- 📖 Gao, L. et al. 2020. *Understanding BERT Rankers Under Distillation*.
- 📖 Hofstätter, S. et al. 2021. *Efficiently Teaching an Effective Dense Retriever with Balanced Topic Aware Sampling*.

🔑 Train a fast student model to mimic a powerful (but slow) teacher

THE IDEA

- 📖 Ding, Y.Q.Y. et al. 2020. *RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering*. NAACL.
- 📖 Gao, L. et al. 2020. *Understanding BERT Rankers Under Distillation*.
- 📖 Hofstätter, S. et al. 2021. *Efficiently Teaching an Effective Dense Retriever with Balanced Topic Aware Sampling*.

🔧 Train a fast student model to mimic a powerful (but slow) teacher

Step 1: Retrieve candidates

Use a fast retriever (BM25, sparse model, or dense index)

```
query: "best pizza in NYC"
candidates: [d1, d2, d3, d4, d5, ...] (top-k from BM25)
```

Step 2: Re-score with teacher

Use a powerful model (cross-encoder, or LLM with zero-shot)

```
teacher scores: [d1: 0.92, d2: 0.31, d3: 0.87, d4: 0.12, d5: 0.65, ...]
```

Step 3: Train student

Resulting dataset for training:

```
(q, [(d1, 0.92), (d2, 0.31), (d3, 0.87), (d4, 0.12), ...])
```

 Hofstätter, S. et al. 2021. Efficiently Teaching an Effective Dense Retriever with Balanced Topic Aware Sampling.

MARGIN-MSE

 Match the margin between positive and negative pairs

$$\mathcal{L} = |(t^+ - t^-) - (s^+ - s^-)|^2$$

where t^+, t^- are teacher scores, s^+, s^- are student scores

KL DIVERGENCE

 Match the full score distribution

$$\mathcal{L} = \text{KL}(\text{softmax}(t/\tau) \parallel \text{softmax}(s/\tau))$$

with temperature τ to soften distributions

We have seen how train an IR model...
... but how to pre-train it in the first place?

Better pre-training improves performance

📖 Gao, L. and Callan, J. 2021. *Condenser: A Pre-training Architecture for Dense Retrieval*.

🔧 Force CLS to encode semantic information

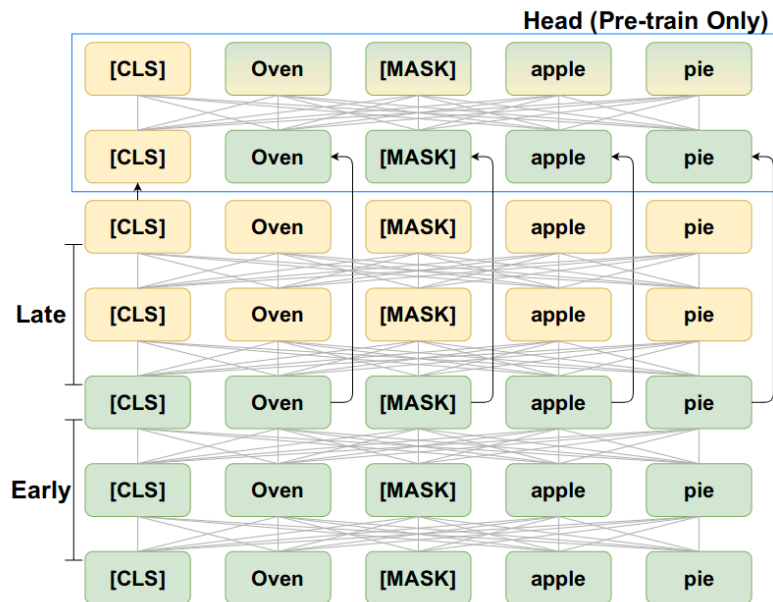


Figure 1: Condenser: Shown are 2 early and 2 late backbone layers. Our experiments each have 6 layers. Condenser Head is dropped during fine-tuning.

🔧 Use document chunks as pseudo-queries

then form a training batch of coCondenser. Write a span s_{ij} 's corresponding *late* CLS representation h_{ij} , its corpus-aware contrastive loss is defined over the batch,

$$\mathcal{L}_{ij}^{co} = -\log \frac{\exp(\langle h_{i1}, h_{i2} \rangle)}{\sum_{k=1}^n \sum_{l=1}^2 \mathbb{I}_{ij \neq kl} \exp(\langle h_{ij}, h_{kl} \rangle)} \quad (6)$$

Familiar readers may recognize this as the contrastive loss from SimCLR (Chen et al., 2020), for which we use random span sampling as augmentation. Others may see a connection to noise contrastive estimation (NCE). Here we provide an NCE narrative. Following the spirit of the distribu-

 *RetroMAE: Pre-Training Retrieval-oriented Language Models Via Masked Auto-Encoder: 2022. <http://arxiv.org/abs/2205.12035>.*

KEY IDEAS

1. **Asymmetric encoder-decoder:** Full BERT encoder, 1-layer decoder
2. **Asymmetric masking:** Moderate for encoder (15-30%), aggressive for decoder (50-70%)
3. **MAE workflow:** Encoder sees masked input → produces embedding → decoder reconstructs original

WHY IT WORKS

- Encoder must compress all information into the embedding (aggressive decoder masking)
- Forces the [CLS] representation to be semantically rich
- Simple to implement, achieves SOTA on MS MARCO and BEIR

ARCHITECTURE

```
Input: "The cat sat on the mat"
Encoder input (30% masked): "The [M] sat on the [M]" → embedding e
Decoder input (70% masked): "[M] [M] [M] on [M] mat" + e → reconstruct original
```

- 🔑 Careful attention to sampling (ranking vs re-ranking)
- 🔑 Weak supervision possible (user interactions and/or classical models)
- 🔑 Distillation is **very effective**
- 🔑 Beware of transfer problems
 - 📖 Thakur, N. et al. 2021. *BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models*.
 - 📖 Zhan, J. et al. 2022. *Evaluating Interpolation and Extrapolation Performance of Neural Retrieval Models*. *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*.
- 🔑 Can learn while indexing (Product Quantization)
 - 📖 Zhan, J. et al. 2021. *Learning Discrete Representations via Constrained Clustering for Effective and Efficient Dense Retrieval*.

CONCLUSION

- **RAG Framework**
 - Grounds generation in retrieved facts
 - Reduces hallucinations
 - Enables access to private/recent information

- **RAG Framework**
 - Grounds generation in retrieved facts
 - Reduces hallucinations
 - Enables access to private/recent information
- **Neural IR Models**
 - Dense models: fast but less precise
 - Cross-encoders: accurate but expensive
 - ColBERT: good balance with indexing capability
 - Sparse models: leverage inverted indices

- **RAG Framework**
 - Grounds generation in retrieved facts
 - Reduces hallucinations
 - Enables access to private/recent information
- **Neural IR Models**
 - Dense models: fast but less precise
 - Cross-encoders: accurate but expensive
 - ColBERT: good balance with indexing capability
 - Sparse models: leverage inverted indices
- **Training & Deployment**
 - Distillation from cross-encoders is effective
 - Two-stage retrieval is standard practice
 - Choose index based on scale and accuracy needs